Maintaining Leiden Communities in Large Dynamic Graphs

Chunxu Lin
CUHK-Shenzhen
Shenzhen, China
chunxulin1@link.cuhk.edu.cn

Yumao Xie
CUHK-Shenzhen
Shenzhen, China
yumaoxie@link.cuhk.edu.cn

Yixiang Fang
CUHK-Shenzhen
Shenzhen, China
fangyixiang@cuhk.edu.cn

Yongmin Hu
ByteDance
Hangzhou, China
huyongmin@bytedance.com

Yingqian Hu
ByteDance
Hangzhou, China
huyingqian@bytedance.com

Cheng Chen
ByteDance
Singapore, Singapore
chencheng.sg@bytedance.com

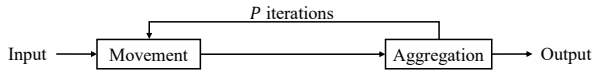
ABSTRACT

Community detection is a foundational capability in large-scale industrial graph analytics, powering applications such as fraud-ring discovery, recommendation systems, and hierarchical indexing for retrieval-augmented generation. Among modularity-based methods, the Leiden algorithm has been widely adopted in production because it delivers high-quality communities with connectivity guarantees. However, real-world graphs evolve continuously, and timely community updates are needed to keep downstream features and retrieval indices fresh. Meanwhile, existing dynamic Leiden approaches recompute the communities whenever their vertices and edges change, thereby almost degrading to near-full recomputation under frequent updates. To alleviate the efficiency issue, we study the efficient maintenance of Leiden communities in large dynamic graphs and present a novel algorithm, called **Hierarchical Incremental Tree Leiden** (`HIT-Leiden`). We first provide a boundedness analysis showing that prior incremental Leiden methods can incur essentially unbounded work even for small updates. Guided by this analysis, we propose `HIT-Leiden` which effectively reduces the range of affected vertices by maintaining connected components and hierarchical community structures. Extensive experiments on large real-world dynamic graphs demonstrate that `HIT-Leiden` achieves community quality comparable to the state-of-the-art competitors while delivering speedups of up to five orders of magnitude over existing solutions. The production deployment results show that `HIT-Leiden` meets stringent latency requirements under high-rate updates at scale.

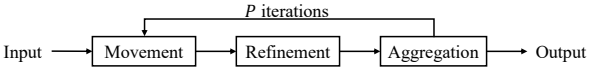
PVLDB Reference Format:
Chunxu Lin, Yumao Xie, Yixiang Fang, Yongmin Hu, Yingqian Hu, and Cheng Chen. Maintaining Leiden Communities in Large Dynamic Graphs. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

1 INTRODUCTION

In ByteDance applications such as Douyin (the Chinese version of TikTok), Toutiao, and Lemon8[1], they generate graph data with billions of nodes and hundreds of billions of edges every day. Many critical tasks rely on grouping vertices into cohesive communities, such as fraud-ring discovery [26, 36, 58, 74], recommendation systems [49, 70, 70], and hierarchical indexing in retrieval-augmented generation (RAG) systems [41, 48, 56, 95]. These industrial applications have two practical requirements: (1) *high-quality* communities that are structurally meaningful and stable for downstream consumption, and (2) *low-latency updates* under continuous graph evolution. For example, for the graph of risk monitoring in ByteDance, a single update batch can contain up to 10^5 edge updates. Yet, the community structure is expected to be refreshed within a few minutes, rather than hours or days required for a full rerun. Such delays cause features and retrieval indices to be outdated, thereby degrading end-to-end performance.



(a) The process of the Louvain algorithm [9].



(b) The process of the Leiden algorithm [72].

Figure 1: Illustrating the Louvain and Leiden algorithms.

To detect network communities, the modularity-based community detection methods, such as Louvain [9] and Leiden [72] which aim to maximize the modularity, have been widely adopted in industry because of their high-quality communities. As a well-known community detection metric, modularity measures the difference between the actual number of edges in a community and the expected number of such edges. However, the Louvain algorithm may produce *internally disconnected communities*, which can mix unrelated components and degrade downstream utility in production pipelines. As illustrated in Figure 1, the Louvain algorithm iterates the *movement* and *aggregation* phases,in which vertices are moved to the community that yields the largest increase in modularity, and then communities are aggregated into supervertices for the next iteration. The Leiden algorithm retains the above two iterative phases of the Louvain algorithm, but introduces an iterative *refinement* phase that splits potentially disconnected communities into connected sub-communities before aggregation. As shown in Figure 2, the refinement phase can split a disconnected community after the movement phase into multiple connected sub-communities, and then aggregate these sub-communities into a supergraph to repeat the process. The refinement phase is crucial for quality and connectivity.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

[1] https://www.douyin.com/; https://www.toutiao.com/; https://www.lemon8-app.com/

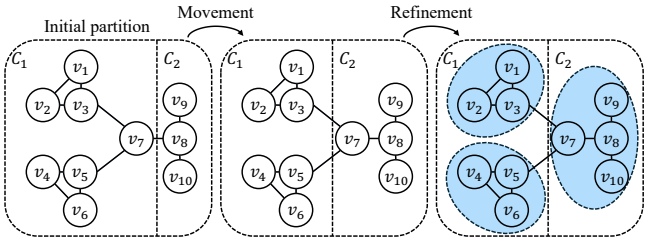
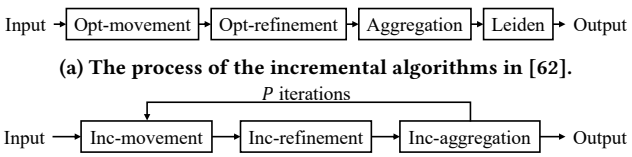


Figure 2: An example of the movement and refinement phases in the Leiden algorithm.

Industrial motivation. In ByteDance, many graph analytics tasks are built on top of the *Leiden* algorithm. As real-world graphs evolve continuously, recomputing communities from scratch is often too costly to meet production service-level agreements, which creates a strong need for efficient maintenance of Leiden communities in large dynamic graphs. We highlight two representative such tasks in ByteDance. (1) *Anomaly detection:* This task's pipeline operates on the graph with 10^8 vertices and 2×10^9 edges, where recomputing Leiden communities from scratch introduces the delay of several hours. However, the online monitoring and operational workflows require the incremental maintenance for batches of roughly 10^5 edge updates within a few minutes. (2) *Video recommendation:* This task works on a graph with 10^9 vertices and 4×10^{10} edges. Full recomputation of Leiden communities incurs day-level delays, while product requirements demand hour-level updates even under billion-scale edge changes per batch.

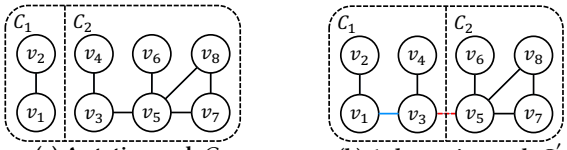


(a) The process of the incremental algorithms in [62].

(b) The process of our `HIT-Leiden` algorithm.

Figure 3: Algorithms for maintaining Leiden communities.

Limitations of prior work. Despite several incremental algorithms for maintaining Louvain communities on dynamic graphs (e.g., [17, 61, 89, 96]), little attention has been paid to incremental Leiden maintenance. These Louvain-based techniques do not directly extend to Leiden because the Leiden algorithm introduces an additional refinement phase that enforces the well-connectedness of communities. To the best of our knowledge, [62] is the only work that targets this problem. As shown in Figure 3(a), all of its approaches optimize only the first iteration of the Leiden algorithm (movement and refinement) and then invoke the original static Leiden algorithm for the remaining iterations. However, wen a vertex or edge is inserted or deleted, the local change may force the refinement to be re-evaluated and rerun, which further cascades through the movement phase and hierarchy, making the affected region hard to bound and leading to unpredictable latency that can approach full recomputation. For example, in Figure 4, the approaches in [62] may rerun the refinement phase for all communities, effectively processing the whole graph even for minor updates. Moreover, the prior work lacks boundedness analysis that explains when and why incremental maintenance remains locally,



(a) A static graph G **(b) A dynamic graph G'**

Figure 4: Illustrating community maintenance, where (v_1, v_3) is a newly inserted edge and (v_3, v_5) is a newly deleted edge.

thereby leaving practitioners without guidance on worst-case behavior and latency risks. Hence, it is desirable to develop a novel efficient algorithm to maintain Leiden communities.

Our approach and contributions. This paper presents Hierarchical Incremental Tree Leiden (`HIT-Leiden`), a practical dynamic community maintenance algorithm designed for industrial-scale, frequently updated graphs. Our key insight is that Leiden inherently induces a tree-like hierarchical structure across iterations. By explicitly maintaining this hierarchy and the connectivity structure within the affected regions, we can significantly reduce the scope of recomputation and stabilize update latency. Figure 3(b) outlines our design: `HIT-Leiden` performs *inc-movement*, *inc-refinement*, and *inc-aggregation* iteratively on supergraphs whose vertices correspond to sub-communities, thereby exploiting hierarchical dependencies rather than operating solely at the original-vertex level. Inspired by dynamic connectivity maintenance techniques [82], *inc-refinement* uses tree-based structures to efficiently update sub-communities while preserving connectivity guarantees.

Concretely, we make the following contributions: (1) We provide a boundedness analysis for existing incremental Leiden approaches, showing why their maintenance cost can be unbounded, and thus their cost may be extremely high; (2) We propose `HIT-Leiden`, which incrementally maintains both hierarchical community structures and connected sub-communities across supergraph levels; (3) We evaluate `HIT-Leiden` on multiple large real-world dynamic graph datasets and show that it achieves community quality comparable to the state-of-the-art incremental Leiden methods while being up to five orders of magnitude faster than them. (4) We deploy `HIT-Leiden` in production at ByteDance, demonstrating that it meets stringent latency requirements on large-scale graphs under high-rate updates.

Outline. We review related work in Section 2. Section 3 introduces preliminaries and the problem definition. Section 4 presents our analysis. Section 5 describes `HIT-Leiden`. We report experimental results in Section 6 and conclude in Section 7.

2 RELATED WORK

In this section, we first review the existing works of CD for both static and dynamic graphs. We simply classify these works as modularity and other metrics-based CD methods.

• **Modularity-based CD.** Modularity-based CD methods aim to partition a graph such that communities exhibit high internal connectivity relative to a null model. Among these methods, Louvain [9] is the most popular one due to its high efficiency and scalability as shown in some comparative analyses [4, 37, 86]. Leiden [72] improves upon Louvain by resolving the problem of disconnected communities, yielding higher-quality results with comparable runtime. Other modularity heuristics [18, 50, 52] or incorporate simulated

annealing [10, 35], spectral techniques [53], and evolutionary strategies [40, 43]. Further refinements explore multi-resolution [68], robust optimization [5], normalized modularity [46], and clustering cost frameworks [33]. Recent neural approaches have integrated modularity objectives into deep learning models [8, 11, 81, 85, 91], enhancing representation learning for CD.

Besides, some recent works have studied how to incrementally maintain modularity-based communities when the graph is changed. Aynaud et al. [6] proposed one of the earliest approaches by reusing previous community assignments to warm-start the Louvain algorithm. Subsequent works extended this idea to both Louvain [17, 19, 47, 55, 61, 65, 66, 89] and Leiden [62], incorporating mechanisms such as edge-based impact screening or localized modularity updates. Nevertheless, the existing algorithms of maintaining Leiden communities lack in-depth theoretical analysis, and their practical efficiency is poor. Other methods based on modularity, including extensions to spectral clustering [16], multi-step CD [7], and label propagation-based methods [54, 78–80] have been studied on dynamic graphs.

• **Other metrics-based CD.** Beyond modularity, various CD methods have been developed by using different optimization purposes, such as similarity, statistical inference, spectral clustering, and neural networks. The similarity-based methods like SCAN [22, 75, 84] identify dense regions from the graph via structural similarity. Statistical inference approaches, including stochastic block models [2, 27, 34, 57], infer communities by fitting generative probabilistic models to observed networks. Spectral clustering methods [3, 21, 51] exploit the eigenstructure of graph Laplacians to group nodes with similar structural roles. Deep learning-based methods for CD have recently gained traction. Graph convolutional networks [20, 29, 30, 38, 44, 67, 83, 92, 94], and graph attention networks [25, 32, 45, 73, 76, 88] have demonstrated strong performance in learning expressive node embeddings for CD tasks. For more details, please refer to recent survey papers of CD [12, 69].

Besides, many of the above methods have also been extended for dynamic graphs. Ruan et al. [60] and Zhang et al. [90] have studied structural graph clustering on dynamic graphs, which is based on structural similarity. Temporal spectral methods [15, 16] and dynamic stochastic block models [42, 64] enable statistical modeling of evolving community structures over time. Recent deep learning approaches also support dynamic CD through mechanisms such as temporal embeddings [93], variational inference [39], contrastive learning [14, 23, 77], and generative modeling [31]. These models capture temporal dependencies and structural evolution.

3 PRELIMINARIES

In this section, we first briefly recap the original Leiden procedure, then briefly introduce the maintenance of Leiden communities problem, and finally introduce the key notations.

3.1 Leiden algorithm

Originally, the Leiden algorithm [72] was designed for static graphs. It mainly consists of three phases: *movement*, *refinement*, and *aggregation*, which are executed iteratively until the overall modularity is maximized. Algorithm 1 presents the algorithm. Starting from the original graph $G^1 = G$ and an initial community mapping $f(\cdot)$,

Algorithm 1: Leiden algorithm [63, 71]

Input: $G, f(\cdot), P, \gamma$
Output: Updated $f(\cdot)$
1 $G^1 \leftarrow G, f^1(\cdot) \leftarrow f(\cdot)$;
2 **for** $p = 1$ *to* P **do**
3 $\quad f^p(\cdot) \leftarrow Move(G^p, f^p(\cdot), \gamma)$;
4 $\quad s^p(\cdot) \leftarrow Refine(G^p, f^p(\cdot), \gamma)$;
5 $\quad$ **if** $p < P$ **then**
6 $\quad\quad G^{p+1}, f^{p+1}(\cdot) \leftarrow Aggregate(G^p, f^p(\cdot), s^p(\cdot))$;
7 Update $f(\cdot)$ using $s^1(\cdot), \cdots, s^P(\cdot)$;
8 **return** $f(\cdot)$;

Leiden iterates for P rounds (lines 1-2) where P is a pre-defined parameter that users often set in practice to balance modularity gains against runtime. In each round p, the *movement* phase relocates vertices (or supervertices) to improve modularity, producing an updated mapping $f^p(\cdot)$ from the supervertices (or supervertices) in this iteration to communities (line 3). The *refinement* phase splits each community into connected sub-communities, producing a sub-community mapping $s^p(\cdot)$ from the supervertices in this iteration to the supervertices in the next iteration (line 4). The *aggregation* phase contracts each sub-community into a supervertex to form the next-level supergraph G^{p+1} (lines 5-6). After P rounds, the final partition $f(\cdot)$ of G is obtained by composing the mappings across levels.

The overall time complexity of Algorithm 1 is $O(P \cdot (|V| + |E|))$, since each iteration costs $O(|V| + |E|)$ time [63], where V and E are the vertex set and edge set of G, respectively.

Example 1. *Figure 6 (a) depicts the process of Leiden with P=3 for the graph in Figure 4. Denote by v_i^p the supervertex (i.e., sub-community) in the p-th iteration of Leiden. It generates three levels of supergraphs: G^1, G^2, and G^3, with $G^1 = G$. The vertices of these supergraphs form a tree-like structure, as shown in Figure 6(b).*

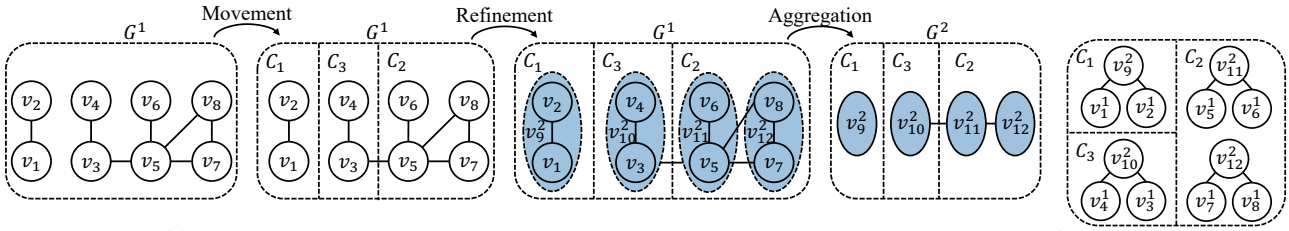
Take the first iteration as an example depicted in Figure 5. In the movement phase, it generates three communities $C_1 = \{v_1^1, v_2^1\}$, $C_2 = \{v_5^1, v_6^1, v_7^1, v_8^1\}$ and $C_3 = \{v_3^1, v_4^1\}$. In the refinement phase, C_2 is split into two sub-communities $v_{11}^2 = \{v_5^1, v_6^1\}$ and $v_{12}^2 = \{v_7^1, v_8^1\}$, and C_1 and C_3 are unchanged. In the aggregation phase, all vertices are aggregated into supervertices based on their sub-community memberships, resulting in G^2.

3.2 Maintaining Leiden communities in dynamic graphs

For simplicity, we call the communities generated by the Leiden algorithm *Leiden communities*. The goal of maintaining Leiden communities is to refresh the communities after each graph update. We now formally introduce the problem studied in this paper.

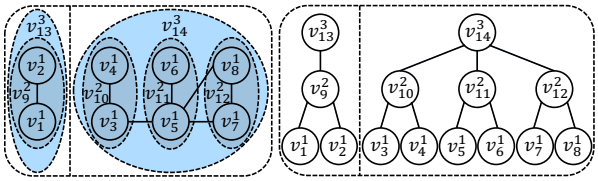
Problem 1 (Maintenance of Leiden communities [62]). *Given a graph G with its Leiden communities $\mathbb{C}$, and some edge updates ΔG, return the updated Leiden communities after applying ΔG to G.*

As shown in Algorithm 1, the Leiden algorithm has three phases. As a result, to maintain Leiden communities when the graph is updated, we can follow the same three-phase procedure, but only make updates in a small affected subgraph: *(1) Movement phase:*



(a) The process of hierarchical partitions at the first iteration on the graph. **(b) The tree-like structure.**

Figure 5: The process of hierarchical partitions in Figure 6 at level 1 under the Leiden algorithm.



(a) All the communities. **(b) A tree-like structure.**

Figure 6: The process of Leiden for the graph G in Figure 4(a).

Table 1: Frequently used notations and their meanings.

Notation	Meaning
$G = (V, E)$	A graph with vertex set V and edge set E
$N(v)$, $N_2(v)$	The vertex v's 1- and 2-hop neighbor sets, resp.
$w(v_i, v_j)$	The weight of the edge between v_i and v_j
$d(v)$	The weighted degree of vertex v
m	The total weight of all edges in G
$\mathbb{C}$	A set of communities forming a partition of G
Q	The modularity of the graph G with partition $\mathbb{C}$
$G^p = (V^p, E^p)$	The supergraph in the p-th iteration of Leiden with vertex set V^p and edge set E^p
$\Delta Q(v \to C', \gamma)$	Modularity gain by moving v from C to C' given γ
$f(\cdot): V \to \mathbb{C}$	A mapping from vertices to communities
$f^p(\cdot): V^p \to \mathbb{C}$	A mapping from supervertices to communities
$s^p(\cdot)$: $V^p \to V^{p+1}$	A mapping from supervertices at level p to supervertices at level $(p+1)$ (sub-communities)
V^p, E^p	A set of supervertices and superedges at level p

we update the community assignments for affected vertices to improve modularity; *(2) Refinement phase:* we restore the connectivity guarantees by splitting and merging affected parts inside each community; *(3) Aggregation phase:* we update the supergraph and propagates structural changes to higher levels when needed.

However, this task is challenging for two reasons. First, the refinement phase enforces intra-community connectivity, which requires efficiently tracking connectivity changes under edge insertions or deletions. Second, because the refinement and aggregation phases are tightly coupled, even a small local update may cascade across multiple hierarchical levels, leading to unstable and potentially large maintenance costs.

3.3 Notations

We consider an **undirected weighted graph** $G = (V, E)$, where V and E denote the vertex and edge sets, respectively. The function $w(\cdot)$ maps each edge to a positive weight. For a vertex v, let $N(v)$ be its neighbor set, and let $d(v)$ be its weighted degree. We define m as the sum of all edge weights, and define $d(C) = \sum_{u \in C} d(u)$ as the total weighted degree of community C. A community partition is denoted by $\mathbb{C} = \{C_1, \ldots, C_k\}$, and we use a mapping $f(\cdot) : V \to \mathbb{C}$ to denote the community membership of each vertex. For convenience, Table 1 provides a quick reference for frequently used symbols. For each vertex v, the sum of weights between v and the community C is denoted by $w(v, C)$.

The Leiden algorithm, a renowned heuristic approach in community detection based on modularity, primarily aims to compute a partition that locally maximizes modularity.

Definition 1 (Modularity gain [9]). *Given a graph G, a partition $\mathbb{C}$, and a vertex v that belongs to a community C, the modularity gain of moving v from C to another community C' is defined as:*

$$\Delta Q(v \to C', \gamma) = \frac{w(v, C') - w(v, C)}{2m} + \frac{\gamma \cdot d(v) \cdot (d(C) - d(v) - d(C'))}{(2m)^2}. \tag{1}$$

In this paper, we focus on dynamic graphs with insertions and deletions of both vertices and edges. Since a vertex insertion (resp. deletion) can be modeled as a sequence of edge insertions (resp. deletions), we simply focus on edge changes. Given a set of edge changes ΔG for a graph $G = (V, E)$, we obtain an updated graph $G' = (V', E')$ by applying ΔG to G. Since there are two types of edge updates, we let ΔG be the set of edge updates, which contains edge insertions (v_i, v_j, α) and deletions $(v_i, v_j, -\alpha)$, where $\alpha > 0$. Given P iterations, we denote the supergraph in the p-th iteration as $G^p = (V^p, E^p)$.

4 THEORETICAL ANALYSIS OF LEIDEN

In this section, we first analyze the boundedness of existing algorithms, then study how vertex behavior impacts community structure under graph updates, and extend our analysis to supergraphs.

4.1 Boundedness analysis

We first introduce some concepts related to boundedness.

• **Concepts of boundedness.** The notion of boundedness [59] evaluates the effectiveness of an incremental algorithm using the metric CHANGED. For the Leiden algorithm, it is a set of vertices whose neighbor edges or community memberships are changed.

Definition 2 (Boundedness [24, 59]). *An incremental algorithm is bounded if its computational cost can be expressed as a polynomial function of* |CHANGED|. *Otherwise, it is unbounded.*

• **Concept of relative boundedness.** In real-world dynamic graphs, |CHANGED| is often small, yet some unbounded algorithms can be solved in polynomial time using measures comparable to |CHANGED|, making these algorithms feasible. To assess these incremental algorithms effectively, Fan et al. [24] introduced the concept of relative boundedness, which leverages a more refined cost model

called the affected region. Let AFF denote the affected region, the part of the graph actually processed by the incremental algorithm.

Definition 3 (AFF [24]). *Given a graph G, an updated graph G', and a query,* AFF *signifies the difference between the costs incurred by the static algorithm when computing the query on G versus on G'.*

Unlike CHANGED, AFF captures the concrete portion of the graph touched by an incremental algorithm, providing a tighter bound on its computational cost. We define the vertices and the supervertices that change their communities or sub-communities as the affected region AFF for Leiden. This leads to the following definition.

Definition 4 (Relative boundedness [24]). *An incremental graph algorithm is relatively bounded with respect to the static algorithm if its cost is polynomial in* |AFF|.

We now analyze the boundedness of existing incremental Leiden algorithms.

Theorem 1. *When processing an edge update, the incremental Leiden algorithms proposed in [62] all cost $O(P \cdot (|V| + |E|))$.*

Table 2: Incremental Leiden algorithms

Method	Time complexity	Relative boundedness
ND-Leiden [62]	$O(P \cdot (\lvert V\rvert + \lvert E\rvert))$	✗
DS-Leiden [62]	$O(P \cdot (\lvert V\rvert + \lvert E\rvert)))$	✗
DF-Leiden [62]	$O(P \cdot (\lvert V\rvert + \lvert E\rvert))$	✗
HIT-Leiden	$O(\lvert N_2(\texttt{CHANGED})\rvert + \lvert N_2(\texttt{AFF})\rvert)$	✓

By Theorem 1, the existing algorithms for maintaining Leiden communities are both unbounded and relatively unbounded, as shown in Table 2. They are very costly for large graphs, even with a small update. Following, we review the properties of Leiden.

4.2 Vertex optimality and subpartition γ-density

As shown in the literature [9, 72], if $s^P(\cdot) = f^P(\cdot)$ after P iterations, Leiden is guaranteed to satisfy the following two properties:

- **Vertex optimality:** All the vertices are vertex optimal.
- **Subpartition γ-density:** All the communities are subpartition γ-dense.

To design an efficient and effective maintenance algorithm for Leiden communities, we analyze how vertices and communities behave under graph changes, as follows.

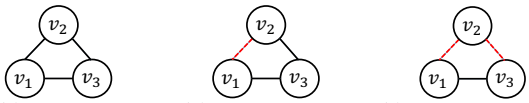
• **Analysis of vertex optimality.** We begin with a key concept.

Definition 5 (Vertex optimality [9]). *A community $C \in \mathbb{C}$ is called vertex optimal if moving any vertex v from its community C to any other community C' does not increase the modularity of $\mathbb{C}$.*

Next, we introduce an assumption in the maintenance of Louvain communities [61, 89], which is strongly supported by our observations from industrial workloads as discussed in Section 1.

Assumption 1. *The sum of weights of the updated edges is sufficiently small relative to the total edge weight m of the graph.*

Based on Assumption 1, prior studies suggest that when the number of edge updates is small relative to the graph size, three heuristics hold: (1) Intra-community edge deletions and inter-community edge insertions could affect vertex-level community memberships [61, 89]; (2) Inter-community edge deletions and intra-community edge insertions can be ignored [61, 89]; (3) Vertices directly involved in such edge changes are the most likely to alter their community memberships [61]. The heuristics are stated in Observation 1, which can be proved based on Definition 5.



(a) A triangle. **(b) Delete an edge.** **(c) Delete two edges.**

Figure 7: An example illustrating subpartition γ-density.

Observation 1 ([61]). *Given an intra-community edge deletion or a cross-community edge insertion, its effect on the community memberships cannot be ignored.*

We further derive the propagation of community changes from Observation 1.

Lemma 1. *When a vertex v changes its community to C, then the community memberships of its neighbors not in C in the updated graph could be affected.*

Proof. For lack of space, the proof of the lemma is given in the appendix of the full version [1] of this paper. □

Based on these analyses, we develop a novel movement phase, called inc-movement in HIT-Leiden to preserve vertex optimality, which will be introduced in Section 5.1.

• **Analysis of subpartition γ-density.** To facilitate understanding of subpartition γ-density, we first introduce γ-order and γ-connectivity, which are key concepts for defining subpartition γ-density.

Definition 6 (γ-order). *For two vertex sequences X and Y in a graph, let $X \otimes Y$ denote merging Y into X such that the modularity value of the merged sequence exceeds the sum of the modularity values of the two original sequences. A γ-order of a vertex sequence $U = \{v_1, \cdots, v_x\}$ represents a merged sequence starting from singleton sequences $\{v_1\}, \cdots, \{v_x\}$.*

We can maintain one γ-order per sub-community from Leiden, which is represented by the sequence of vertices merging into the sub-community in refinement of Leiden.

Definition 7 (γ-connectivity [72]). *Given a graph, a vertex sequence U is γ-connected if U can be generated from at least one γ-order.*

Definition 8 (Subpartition γ-density [72]). *A vertex subsequence U in a community C is subpartition γ-dense if U is γ-connected under a γ-order, and removing any intermediate vertex sequence (in the γ-order) from C does not increase the modularity value of the partition.*

Example 2. *The triangle in Figure 7(a) is subpartition γ-dense with $\gamma = 1$ since there are six different γ-orders. For instance, one is $\{v_3\} \otimes (\{v_1\} \otimes \{v_2\})$, which represents that $\{v_2\}$ is merged into $\{v_1\}$ generating the sequence $\{v_1, v_2\}$, and then $\{v_1, v_2\}$ merges into $\{v_3\}$ generating $\{v_1, v_2, v_3\}$. After deleting the edge (v_1, v_2), although $\{v_3\} \otimes (\{v_1\} \otimes \{v_2\})$ is not a γ-order, the updated graph is still subpartition γ-dense since $\{v_1\} \otimes (\{v_2\} \otimes \{v_3\})$ is a γ-order in the updated graph. After further deleting the edge (v_2, v_3), the updated graph is not subpartition γ-dense since v_2 is not connected to v_1 and v_3.*

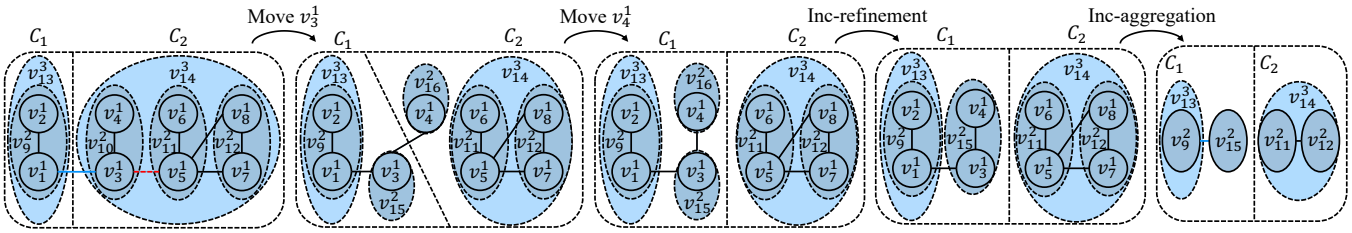
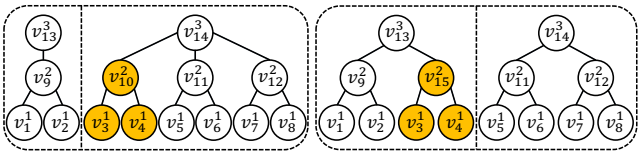


Figure 8: An example of `HIT-Leiden`.



(a) Before maintenance. **(b) After maintenance.**

Figure 9: The hierarchical partition changes of Figure 4.

In essence, each community C (or sub-community S) of Leiden is subpartition γ-dense, since (1) any sub-community in C (or S) is locally optimized, and (2) all sub-communities are γ-connected. Notably, as shown in Figure 3(b), vertex optimality ensures the first condition by design since any sub-community will be a supervertex in `inc-movement` of the next iteration. Thus, we will develop a new refinement algorithm, `inc-refinement`, to preserve γ-connectivity of sub-communities.

Next, we analyze the γ-connectivity property under two kinds of graph updates, i.e., *edge deletion* and *edge insertion*. For lack of space, all detailed discussions regarding Observations 2-4 are shown in the appendix of the full version [1] of this paper.

(1) Edge deletion. We consider the deletions of both intra-sub-community edges and cross-sub-community edges:

Observation 2. *Given an intra-sub-community edge deletion with a γ-order, the endpoint of the edge, which is later merged into the sub-community according to the γ-order, could leave the sub-community. Its effect on the earlier-merged endpoint of the edge can be ignored.*

Observation 3. *Given a cross-sub-community edge deletion, its effect on sub-community memberships can be ignored.*

(2) Edge insertion. We consider both intra-sub-community edge insertion and cross-sub-community edge insertion:

Observation 4. *Given an intra-sub-community or a cross-sub-community edge insertion, it is unlikely to affect the sub-community memberships.*

By the above observations, when designing the maintenance algorithm, we only need to consider the effect of vertices that are the later-merged endpoints in intra-sub-community edge deletions, which indicates that this case can also be ignored. Furthermore, our experiments also revealed the following observation, indicating that this case can also be ignored.

Observation 5. *Given an updated graph with its previous sub-community memberships, for any sub-community S, we treat each connected component in S as a new sub-community. Most of the maintained communities are subpartition γ-dense.*

The above observation holds because Leiden only provides a γ-order from the refinement phase, and a subgraph often exists with multiple distinct γ-orders as shown in Example 2. Besides, if a vertex is a candidate affecting γ-connectivity, it is often a candidate affecting vertex optimality, e.g., the vertex v_2 in Figure 7(c). In this case, the vertex is likely to change its community before verifying whether the vertex needs to move out of its sub-community. Hence, the case (1) of Observation 2 can be ignored if the intra-sub-community edge deletion does not cause the sub-community to be disconnected.

Based on Observation 5, we develop a novel refinement algorithm, called `inc-refinement`, in `HIT-Leiden`, which will be introduced in Section 5.2. As will be shown in Section 6.2 (Figure 14 and Figure 15(b)), over 99% of the maintained communities from `HIT-Leiden` are subpartition γ-dense.

Extension to supergraphs. Changes at the lower level propagate upward to superedge changes in the higher-level supergraph, as Leiden constructs a list of supergraphs in a bottom-up manner. This motivates us to develop an incremental aggregation phase, namely `inc-aggregation`, as described in Section 5.3.

Example 3. *In Figure 4, communities C_1 and C_2 are treated as supervertices. Deleting an edge $(v_3, v_5, 1)$ and inserting an edge $(v_1, v_3, 1)$ cause v_3 and v_4 to move from C_2 to C_1. This results in the deletion of $(C_2, C_2, -2)$ and the insertion of $(C_1, C_1, 2)$ in the supergraph.*

Therefore, we treat each supergraph as a set of facing edge changes from the previous Leiden community and process them using a consistent procedure as shown in Figure 3(b).

Example 4. *Consider the result in Figure 6. The graph undergoes an edge deletion $(v_3^1, v_5^1, -1)$ and an edge insertion $(v_1^1, v_3^1, 1)$. Resulting community and sub-community changes are shown in Figure 8, with hierarchical changes in Figure 9. Take the first iteration as an example. According to Observation 1 and Observation 5, the vertices directly affected by the edge updates and thus processed in `inc-movement` are v_1^1, v_3^1, and v_5^1. No vertices are directly affected in `inc-refinement`. In `inc-movement`, the vertex v_3^1 is first migrated from C_2 to C_1 to increase the modularity, while v_1^1 and v_5^1 do not change their community memberships. Subsequently, based on Lemma 1, we need to examine v_4^1 in `inc-movement` and then migrate it from C_2 to C_1. Because this movement breaks the sub-community v_{10}^2, based on Observation 5, we reassigned v_3^1 and v_4^1 to two new sub-communities, respectively. `Inc-refinement` processes two new singleton sub-communities v_{15}^2 and v_{16}^2, and then merges them into a sub-community v_{15}^2. `Inc-aggregation` aggregates sub-communities into supervertices, and computes the hyperedge changes in the next iteration, including edge insertion $(v_9^2, v_{15}^2, 1)$ and edge*

deletion $(v_{10}^2, v_{11}^2, -1)$. *Subsequent iterations proceed identically to the initial iteration.*

5 OUR HIT-LEIDEN ALGORITHM

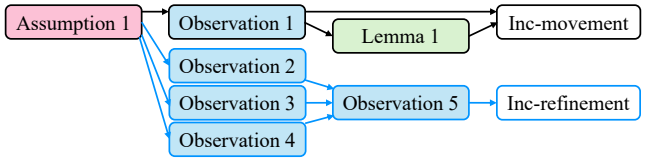


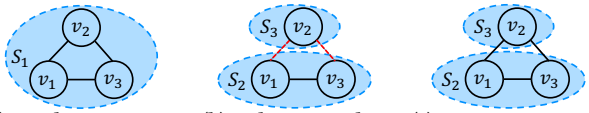
Figure 10: The design rationale for `inc-movement` and `inc-refinement`.

In this section, we propose `HIT-Leiden`, which updates community memberships by restricting computation to the affected region, motivated by the analysis in Section 4.2. We first introduce the three key components, namely `inc-movement`, `inc-refinement`, and `inc-aggregation` in `HIT-Leiden`. Figure 10 shows the assumption, the lemma, and the observations used in these components. Then, we present an auxiliary procedure, called deferred update, abbreviated as `def-update`. Afterward, we give an overview of `HIT-Leiden`, and finally analyze the boundedness of `HIT-Leiden`.

5.1 Inc-movement

The goal of `inc-movement` is to preserve vertex optimality. As analyzed in Section 4.2, the endpoints of a deleted intra-community edge or an inserted cross-community edge may affect their community memberships. If an affected vertex changes its community, its neighbors outside the target community may also be affected. Note that any vertex that changes its community has to change its sub-community, since each sub-community is a subset of its community. Hence, sub-community memberships are also considered in `inc-movement`.

We first introduce the data structures used to maintain a dynamic sub-community. According to Observation 5, each connected component of a sub-community is treated as a γ-connected subset. When edge updates or vertex movements split a sub-community into multiple connected components, we re-assign each resulting component as a new sub-community.



(a) A sub-community. (b) Delete two edges. (c) Move out a vertex.

Figure 11: Illustrating the process that a sub-community S_1 is split into two sub-communities S_1 and S_2.

Example 5. *Figure 11 shows the sub-community S_1 is split into two sub-communities $S_2 = \{v_1, v_3\}$ and $S_3 = \{v_2\}$. The separation can occur either due to the deletion of edges (v_1, v_2) and (v_2, v_3) during graph updates, as shown in Figure 11(b), or due to the removal of vertex v_2 during the movement phase, as shown in Figure 11(c).*

To preserve the structure under such changes, we leverage dynamic connected component maintenance techniques. Various index-based methods have been proposed for this purpose, such as D-Tree [13], DND-Tree [82], and HDT [28]. Let Ψ denote a connected component index, abbreviated as CC-index. The graph G_Ψ stores the subgraph of G consisting only of intra-sub-community edges based on $s(\cdot)$.

Algorithm 2: `Inc-movement`

Input: $G, \Delta G, f(\cdot), s(\cdot), \Psi, \gamma$
Output: Updated $f(\cdot), \Psi, B, K$

1 $A \leftarrow \emptyset, B \leftarrow \emptyset, K \leftarrow \emptyset$;
2 **for** $(v_i, v_j, \alpha) \in \Delta G$ **do**
3 **if** $\alpha > 0$ *and* $f(v_i) \neq f(v_j)$ **then**
4 $A.add(v_i); A.add(v_j)$;
5 **if** $\alpha < 0$ *and* $f(v_i) = f(v_j)$ **then**
6 $A.add(v_i); A.add(v_j)$;
7 **if** $s(v_i) = s(v_j)$ *and* $update_edge\left(G_\Psi, (v_i, v_j, \alpha)\right)$ **then**
8 $K.add(v_i); K.add(v_j)$;
9 **while** $A \neq \emptyset$ **do**
10 $v_i \leftarrow A.pop()$;
11 $C^* \leftarrow argmax_{C \in \mathbb{C} \cup \emptyset} \Delta Q(v_i \rightarrow C, \gamma)$;
12 **if** $\Delta Q(v_i \rightarrow C^*, \gamma) > 0$ **then**
13 $f(v_i) \leftarrow C^*; B.add(v_i)$;
14 **for** $v_j \in N(v_i)$ **do**
15 **if** $f(v_j) \neq C^*$ **then**
16 $A.add(v_j)$;
17 **for** $v_j \in N(v_i) \wedge s(v_i) = s(v_j)$ **do**
18 **if** $update_edge\left(G_\Psi, (v_i, v_j, -w(v_i, v_j))\right)$ **then**
19 $K.add(v_i); K.add(v_j)$;
20 **return** $f(\cdot), \Psi, B, K$;

Algorithm 2 shows `inc-movement`. Given an updated graph G, a set of graph changes ΔG, community mappings $f(\cdot)$, sub-community mappings $s(\cdot)$, and a CC-index Ψ, it first initializes three empty sets: A, B and K (line 1). Here, A keeps the vertices whose community memberships may be changed, B keeps the vertices that have changed their community memberships, and K records the endpoints on edges whose deletion disconnects the connected component in G_Ψ. Subsequently, vertices involved in intra-community edge deletion or cross-community edge insertion are added to A, and edges in G_Ψ are updated according to intra-sub-community changes (lines 2-7) based on Observations 1 and 5, respectively. If an edge update in G_Ψ causes a connected component to split (i.e., $update_edge(\cdot)$ returns *true*), its endpoints are added to K (line 8). It then processes vertices in A until the set is empty (line 9). For each vertex v_i, it identifies the target community C^* that yields the highest modularity gain (lines 10-11). If $\Delta Q(v_i \rightarrow C^*) > 0$, $f(v_i)$ is updated to C^*, v_i is added into B, and the neighbors of v_i not in C^* are added to A (lines 12-16), which implements the property in Lemma 1. Besides, the intra-sub-community edges involving v_i are deleted from G_Ψ, and the vertices involved in component splits are added to K (lines 17-19). Finally, it returns $f(\cdot)$, Ψ, B, and K (line 20).

5.2 Inc-refinement

As discussed in Section 5.1 and Observation 5, we treat each connected component in G_Ψ maintained in `inc-movement` as a sub-community. Therefore, we design `inc-refinement` for re-assigning each new connected component in G_Ψ as a sub-community. Additionally, we attempt to merge singleton sub-communities whose

Algorithm 3: `Inc-refinement`

Input: $G, f(\cdot), s(\cdot), \Psi, K, \gamma$
Output: Updated $s(\cdot), \Psi, R$,
1 $R \leftarrow \emptyset$;
2 **for** $v_i \in K$ **do**
3 Map all vertices in the connected component into a new sub-community and add them into R;
4 **for** $v_i \in R$ **do**
5 **if** v_i *is in a singleton sub-community* **then**
6 $\mathcal{T} \leftarrow \{s(v) | v \in N(v_i) \cap f(v_i), \Delta Q(s(v) \rightarrow \emptyset, \gamma) \leq 0\}$;
7 $S^* \leftarrow argmax_{S \in \mathcal{T}} \Delta M(v_i \rightarrow S, \gamma)$;
8 **if** $\Delta M(v_i \rightarrow S^*, \gamma) > 0$ **then**
9 $s(v_i) \leftarrow S^*$;
10 **for** $v_j \in N(v_i)$ **do**
11 **if** $s(v_i) = s(v_j)$ **then**
12 $update_edge\ (G_\Psi, (v_i, v_j, w(v_i, v_j)))$;
13 **return** $s(\cdot), \Psi, R$;

process is the same as the process of the refinement phase in `Leiden` with G_Ψ maintenance.

Algorithm 3 presents its pseudocode. Given an updated graph G, community mappings $f(\cdot)$ and sub-community mapping $s(\cdot)$, a CC-index Ψ, and a set K, it first initializes R as an empty list to track vertices that have changed their sub-communities (line 1). Note that R is an ordered list sorted in ascending vertex degree. It then traverses K to identify split connected components in G_Ψ using breadth-first search or depth-first search. All vertices in the split connected component are re-mapped to a new sub-community, and added to R (lines 2-3). For each vertex $v_i \in R$ that is in a singleton sub-community, `inc-refinement` uses a set $\mathcal{T}$ to store the locally optimized neighboring sub-communities of v_i within the same community (lines 4-6). Then, it attempts to re-assign v_i to a sub-community $S^* \in \mathcal{T}$, which offers the highest modularity gain to eliminate singleton sub-communities (line 7). Notably, $\Delta M(v_i \rightarrow S, \gamma)$ denotes the modularity gain of moving v_i from $s(v_i)$ to S, whose calculation follows the same formula as the standard modularity gain. If the gain is positive, $s(v_i)$ is updated to S^*, and the corresponding intra-sub-community edges are inserted into G_Ψ (lines 8-12). Finally, `inc-refinement` returns the $s(\cdot)$, Ψ, and R (line 13).

5.3 Inc-aggregation

Given an updated graph G and its edge changes ΔG, modifications to edges and sub-community memberships are reflected as changes to superedges and supervertices in the supergraph H. Let $s_{pre}(\cdot)$ (resp. $s_{cur}(\cdot)$) denotes the vertex-to-supervertex mappings before (resp. after) `inc-refinement`. Any edge change (v_i, v_j, α) in ΔG corresponds to a superedge change $(s_{pre}(v_i), s_{pre}(v_j), \alpha)$ in H, since the weight of a superedge is the sum of weights of edges between their sub-communities. Besides, a vertex v migration from $s_{\text{pre}}(v)$ to $s_{\text{cur}}(v)$ requires updating these weights. Specifically, the original sub-community $s_{pre}(v)$ must decrease the superedge weights corresponding to the edge incident to v, and the new sub-community $s_{cur}(v)$ must increase them under the new assignment.

Algorithm 4: `Inc-aggregation`

Input: $G, \Delta G, s_{pre}(\cdot), s_{cur}(\cdot), R$
Output: $\Delta H, s_{pre}(\cdot)$
1 $\Delta H \leftarrow \emptyset$;
2 **for** $(v_i, v_j, \alpha) \in \Delta G$ **do**
3 $r_i \leftarrow s_{pre}(v_i), r_j \leftarrow s_{pre}(v_j)$;
4 $\Delta H.add((s_i, s_j, \alpha))$;
5 **for** $v_i \in R$ **do**
6 **for** $v_j \in N(v_j)$ **do**
7 **if** $s_{cur}(v_j) = s_{pre}(v_j)$ *or* $i < j$ **then**
8 $\Delta H.add((s_{pre}(v_i), s_{pre}(v_j), -w(v_i, v_j)))$;
9 $\Delta H.add((s_{cur}(v_i), s_{cur}(v_j), w(v_i, v_j)))$;
10 $\Delta H.add((s_{pre}(v_i), s_{pre}(v_i), -w(v_i, v_i)))$;
11 $\Delta H.add((s_{cur}(v_i), s_{cur}(v_i), w(v_i, v_i)))$;
12 **for** $v_i \in R$ **do**
13 $s_{pre}(v_i) \leftarrow s_{cur}(v_i)$;
14 $Compress(\Delta H)$;
15 **return** $\Delta H, s_{pre}(\cdot)$;

Example 6. *Following Example 3, the initial superedge changes due to edge changes are* $(C_1, C_2, 1)$ *and* $(C_2, C_2, -1)$*. Then, vertices* v_3 *and* v_4 *move from* C_2 *to* C_1*, and there are three cases:*

(1) C_1 *gains edges to the neighbors of* v_3*, resulting in two updates:* $(C_1, C_1, 1)$ *and* $(C_1, C_1, 1)$*;*
(2) C_2 *loses edges to the neighbor of* v_3 *are* $(C_1, C_2, -1)$ *and* $(C_2, C_2, -1)$*;*
(3) *The effect of* v_4 *is skipped to avoid duplicate updates, since its only neighbor* v_3 *already changed.*

After compressing the above six superedge changes, we obtain the final superedge changes, which are $(C_1, C_1, 2)$ *and* $(C_2, C_2, -2)$*.*

Algorithm 4 presents `inc-aggregation`. Initially, the set of changes ΔH of H is empty (line 1). Then, it maps the edge changes ΔG to superedge changes using $s_{pre}(\cdot)$ (lines 2-4). Following, it updates superedges for vertices that switch sub-communities by removing edges from the old community and adding edges to the new one. For any vertex v_i in R, if updates superedges with each neighbor v_j if either $s_{cur}(v_j) = s_{pre}(v_j)$ or $i < j$ to avoid duplicate updates (lines 5-9). Besides, it updates the self-loop for the sub-community of v_i (lines 10-11). Finally, it locally updates $s_{pre}(\cdot)$ to match $s_{cur}(\cdot)$ for the next time step (lines 12-13), and compresses entries by summing the weight of identical superedges in ΔH (line 14).

5.4 Overall HIT-Leiden algorithm

Before presenting our overall HIT-Leiden algorithm, we introduce an optimization technique to further improve the efficiency of the vertices' membership update. Specifically, when a supervertex changes its community membership, all the lower-level supervertices associated with it have to update their community membership. As shown in Figure 9, when v_{10}^2 changes its community, v_3^1 and v_4^1 also update their community memberships to the community containing v_{10}^2. However, during the iteration process of `HIT-Leiden`, a supervertex that changes its community does not automatically trigger updates of the community memberships of its constituent lower-level supervertices.

Algorithm 5: `def-update`

Input: $\{f^P(\cdot)\}$, $\{s^P(\cdot)\}$, $\{B^P\}$, P
Output: Updated $\{f^P(\cdot)\}$

1 **for** p *from* P *to* 1 **do**
2 **if** $p \neq P$ **then**
3 **for** $v_i^p \in B^p$ **do**
4 $f^p(v_p^i) = f^{p+1}(s^p(v_p^i))$;
5 **if** $p \neq 1$ **then**
6 **for** $v_i^p \in B^p$ **do**
7 $B^{p-1}.add(s^{-p}(v_i^p))$;
8 **return** $\{f^P(\cdot)\}$;

To resolve the above inconsistency, we perform a post-processing step to synchronize the community memberships across all levels, as described in Algorithm 5. Let $\{B^P\}$ denote a sequence of P sets $\{B^1, \cdots, B^P\}$, $\{s^P(\cdot)\}$ denote a sequence of P adajcent-level supervertex mappings $\{s^1(\cdot), \cdots, s^P(\cdot)\}$, and $\{f^P(\cdot)\}$ denote a sequence of P community mappings $\{f^1(\cdot), \cdots, f^P(\cdot)\}$. Note, each B^p in $\{B^P\}$ collects supervertices at level-p whose community memberships have changed, each $s^p(\cdot)$ in $\{s^P(\cdot)\}$ maps from level-p supervertices to their parent supervertices at level-$(p+1)$, and each $f^p(\cdot)$ in $\{f^P(\cdot)\}$ maps from level-p supervertices to their communities. A supervertex is added to B^p for one of two reasons: (1) it changes its community during `inc-movement`, or (2) its higher-level supervertex changes community. Hence, for each level p, `def-update` updates each supervertex in B^p by re-mapping its community membership of its parent using $s^p(\cdot)$ and $f^{p+1}(\cdot)$ when $p \neq P$ (lines 1-4), and adds its constituent vertices to B^{p-1} for the next level updates where $s^{-p}(\cdot)$ is the inverse mapping of $s^p(\cdot)$ when $p \neq 1$ (lines 5-7). This algorithm also supports updating the mappings $\{g^P(\cdot)\}$ from each level supervertex to its level-P ancestor.

• **Overall HIT-Leiden.** After introducing all the key components, we present our overall `HIT-Leiden` in Algorithm 6. The algorithm proceeds over P hierarchical levels, where each level-p operates on a corresponding supergraph G^p. Besides the community membership $f(\cdot)$, `HIT-Leiden` also maintains supergraphs $\{G^P\}$, community mappings $\{f^P(\cdot)\}$, sub-community mappings $\{g^P(\cdot)\}$, $\{s^P_{pre}(\cdot)\}$ and $\{s^P_{cur}(\cdot)\}$, and CC-indices $\{\Psi^P\}$ to maintain sub-community memberships for each level. Note, $\{s^P_{pre}(\cdot)\}$ are the mappings from the previous time step, and $\{s^P_{cur}(\cdot)\}$ are the in-time mappings to track sub-community memberships as they evolve at the current time step.

Specifically, it initializes $\{s^P_{cur}(\cdot)\} = \{s^P_{pre}(\cdot)\}$. Given the graph change ΔG, it first initializes the first-level update ΔG to ΔG^1 (line 1). It then proceeds through P iterations, each including three phases after updating the supergraph G^p (line 3).

(1) `Inc-movement` (line 4): it re-assigns community memberships of affected vertices to achieve vertex optimality, which yields $f^p(\cdot)$, Ψ, B^p, and K.
(2) `Inc-refinement` (line 5): it re-maps the supervertices of split connected components in Ψ to new sub-communities, producing $s^p_{cur}(\cdot)$, Ψ, and R^p.
(3) `Inc-aggregation` (line 7): it calculates the next level's superedge changes ΔG^{p+1}, and synchronizes $s^p_{pre}(\cdot)$ to match $s^p_{cur}(\cdot)$.

Algorithm 6: `HIT-Leiden`

Input: $\{G^P\}$, ΔG, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$,$\{s^P_{pre}(\cdot)\}$, $\{s^P_{cur}(\cdot)\}$, $\{\Psi^P\}$, P, γ
Output: $f(\cdot)$, $\{G^P\}$, $\{f^P(\cdot)\}$, $\{f^P(\cdot)\}$, $\{s^P_{pre}(\cdot)\}$, $\{s^P_{cur}(\cdot)\}$, $\{\Psi^P\}$

1 $\Delta G^1 \leftarrow \Delta G$;
2 **for** p *from* 1 *to* P **do**
3 $G^p \leftarrow G^p \oplus \Delta G^p$;
4 $f^p(\cdot), \Psi, B^p, K \leftarrow$ `inc-movement`$(G^p, \Delta G^p, f^p(\cdot), s^p_{cur}(\cdot), \Psi, \gamma)$;
5 $s^p_{cur}(\cdot), \Psi, R^p \leftarrow$ `inc-refinement`$(G^p, f^p(\cdot), s^p_{cur}(\cdot), \Psi, K, \gamma)$;
6 **if** $p < P$ **then**
7 $\Delta G^{p+1}, s^p_{pre}(\cdot) \leftarrow$ `inc-aggregation`$(G^p, \Delta G^p, s^p_{pre}(\cdot), s^p_{cur}(\cdot), R^p)$;
8 $\{f^P(\cdot)\} \leftarrow$ `def-update`$(\{f^P(\cdot)\}, \{s^P_{cur}(\cdot)\}, \{B^P\}, P)$;
9 $\{g^P(\cdot)\} \leftarrow$ `def-update`$(\{g^P(\cdot)\}, \{s^P_{cur}(\cdot)\}, \{R^P\}, P)$;
10 $f(\cdot) \leftarrow g^1(\cdot)$;
11 **return** $f(\cdot)$, $\{G^P\}$, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$, $\{s^P_{pre}(\cdot)\}$, $\{s^P_{cur}(\cdot)\}$, $\{\Psi^P\}$;

After P iterations, `def-update` (Algorithm 5) synchronizes community mappings $\{f^P(\cdot)\}$ and sub-community mappings $\{g^P(\cdot)\}$ across levels (lines 8-9). The final output $f(\cdot)$ is set to $g^1(\cdot)$ (line 10). Besides, we also return $\{G^P\}$, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$, $\{s^P_{pre}(\cdot)\}$, $\{s^P_{cur}(\cdot)\}$, and $\{\Psi^P\}$ for the next graph evolution (line 11).

• **Complexity analysis.** We now analyze the time complexity of `HIT-Leiden` over P iterations. Let Γ^p denote the set of supervertices involved in superedge changes, let Λ^p track the supervertices that change their communities or sub-communities at level-p, and let h^p denote the average vertex depth in the spanning tree in [82] at level-p. Therefore, for each level-p, `inc-movement` and `inc-refinement` complete in $O\left(|N_2(\Gamma^p)| + h^p \cdot |\Gamma^p| + |N_2(\Lambda^p)| + h^p \cdot |\Lambda^p|\right)$, and `inc-aggregation` completes in $O(|N(\Gamma^p)| + |N(\Lambda^p)|)$. Besides, the time cost of `def-update` is $O\left(\sum_{p=1}^{P} |\Lambda^p|\right)$. Hence, the total time cost of `HIT-Leiden` can be expressed as $O(|N_2(\texttt{CHANGED})| + |N_2(\texttt{AFF})|)$, as analyzed in Section 4.2. As a result, our `HIT-Leiden` is bounded relative to Leiden. In the worst case, the total time of `HIT-Leiden` is $O\left(\sum_{p=1}^{P} (|V| + |E| + h^p \cdot |E|)\right)$, when all supervertices change their communities. `HIT-Leiden` degenerates to `Leiden` with a full rebuild of CC-indices in [82].

6 EXPERIMENTS

We now present our experimental results. Section 6.1 introduces the experimental setup. Section 6.2 and 6.3 evaluate the effectiveness and efficiency of `HIT-Leiden`, respectively. Section 6.4 discusses the results of two application studies.

6.1 Setup

Datasets. We use four real-world dynamic datasets, including *dblp-coauthor*[2] (academic collaboration), *yahoo-song*[2] (user-song interactions), *sx-stackoverflow*[3] (developer Q&A), and *risk* (financial transactions) provided by ByteDance. All these dynamic edges are

[2] http://konect.cc/networks/
[3] https://snap.stanford.edu/data/

Table 3: Datasets used in our experiments.

Dataset	Abbr.	$\|V\|$	$\|E\|$	Timestamp
dblp-coauthor	DC	1.8M	29.4M	Yes
yahoo-song	YS	1.6M	256.8M	Yes
sx-stackoverflow	SS	2.6M	63.4M	Yes
it-2004	IT	41.2M	1.0B	No
risk	RS	201.0M	4.0B	Yes

associated with real timestamps. We also use one static dataset *it-2004*[4] (a large-scale web graph), but randomly insert or delete some edges to simulate a dynamic graph. All the graphs are treated as undirected graphs. For each real-world dynamic graph, we collect a sequence of batch updates by sorting the edges in ascending order of their timestamps; for *it-2004*, which lacks timestamps, we randomly shuffle its edge order. Table 3 summarizes the key statistics of the above datasets.

Algorithms. We test the following maintenance algorithms:

- `ST-Leiden`: A naive baseline that executes the static Leiden algorithm from scratch when the graph changes.
- `ND-Leiden`: A simple maintenance algorithm in [62], which processes all vertices during the movement phase, initialized with previous community memberships.
- `DS-Leiden`: A maintenance algorithm based on [62], which uses the delta-screening technique [89] to restrict the number of vertices considered in the movement phase.
- `DF-Leiden`: An advanced maintenance algorithm from [62], which adopts the dynamic frontier approach [61] to support localized updates.
- `HIT-Leiden`: Our proposed method.

Dynamic graph settings. As the temporal span varies across datasets (e.g., 62 years for *dblp-coauthor* versus 8 years for *sx-stackoverflow*), we apply a sliding edge window, avoiding reliance on fixed valid time intervals that are hard to standardize. Initially, we construct a static graph using the first 80% of edges. Then, we select a window size $b \in \{10, 10^2, 10^3, 10^4, 10^5\}$, denoting the number of updated edges in an updated batch. Next, we slide this window $r = 9$ times, so we update 9 batches of edges for each dataset. Note that by default, we set $b = 10^3$.

All the algorithms are implemented in C++ and compiled with the gcc 8.3.0 compiler using the same optimization level. We set $\gamma = 1$ and use $P = 10$ iterations. Before running the Leiden community maintenance algorithms, we obtain the communities by running the Leiden algorithm, and `HIT-Leiden` requires an additional procedure to build auxiliary structures. For each dataset, we compute the initial community assignment by running the standard Leiden algorithm on the first batch, and {HIT-Leiden} needs extra processing to build CC-indices. Because the initial partition is not fully stabilized under the capped number of iterations, the first few update batches exhibit a cold-start overhead for all maintenance methods. To avoid this warm-up bias, we report efficiency results excluding the first two batches. Therefore, we exclude the first two batches from efficiency evaluations. Experiments are conducted on a Linux server running Debian 5.4.56, equipped with an Intel(R) Xeon(R) Platinum 8336C CPU @ 2.30GHz and 2.0 TB of RAM.

[4]https://networkrepository.com/

6.2 Effectiveness evaluation

To evaluate the effectiveness of different maintenance algorithms, we compare the modularity value and proportion of subpartition γ-dense communities for their returned communities. We also evaluate the long-term effectiveness of community maintenance.

• **Modularity.** Figure 12 depicts the average modularity values of all the maintenance algorithms, where the batch size ranges from 10 to 10^5. Figure 13 depicts the modularity value across all the 9 batches, where the batch size is fixed as 1,000. Across all datasets, the expected fluctuation in modularity for `ST-Leiden` is around 0.02 due to its inherent randomness. These maintenance algorithms achieve equivalent quality in modularity, since the difference in their modularity values is within 0.01. Overall, our `HIT-Leiden` achieves comparable modularity with other methods.

• **Proportion of subpartition γ-density.** After running `HIT-Leiden`, for each returned community, we try to re-find its γ-order such that any intermediate vertex set in the γ-order is locally optimized, according to Definition 8. If we can find a valid γ-order for a community, we classify it as a subpartition γ-dense community. We report the proportion of subpartition γ-dense communities in Figure 14. The proportions of subpartition γ-dense communities among these Leiden algorithms are almost 1, and they are within the expected fluctuation caused by the inherent randomness of the measure method. Thus, `HIT-Leiden` achieves a comparable percentage of subpartition γ-density with others.

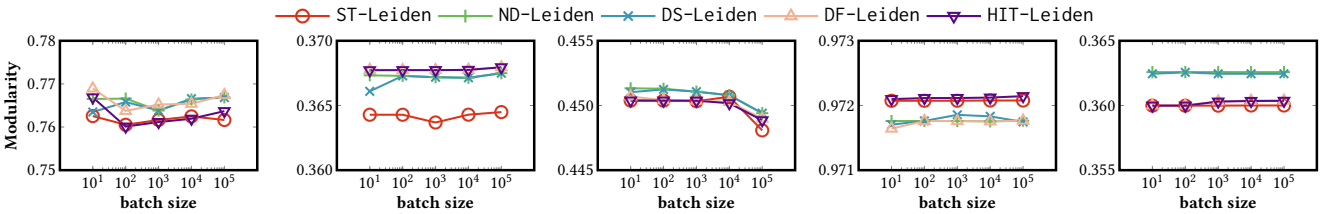
• **Long-term effectiveness.** To demonstrate the long-term effectiveness of maintaining communities, we enlarge the number r of batches from 9 to 999 and set $b = 10,000$. Figure 15 presents the modularity, proportion of subpartition γ-dense communities, and runtime on the sx-stackoverflow dataset. We observe that incremental Leiden algorithms exhibit higher stability than `ST-Leiden` in modularity since they use previous community memberships, and `HIT-Leiden` is faster than other algorithms.

6.3 Efficiency evaluation

In this section, we first present the overall efficiency results, then analyze the time cost of each component, and finally evaluate the effects of some hyperparameters.

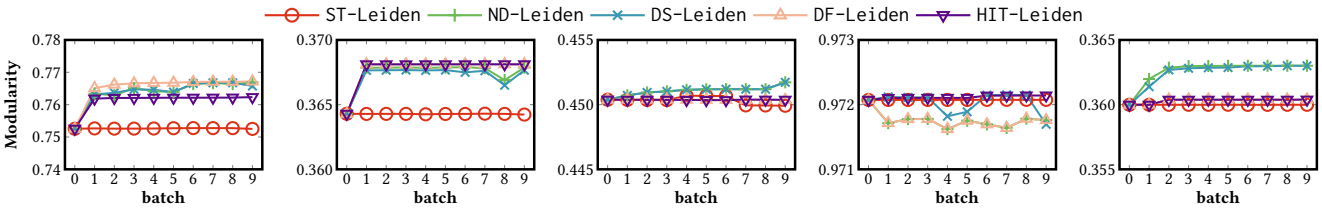
• **Overall results.** Figure 16 presents the overall efficiency results where b is set to its default value $1,000$. Clearly, `HIT-Leiden` achieves the best efficiency on datasets, especially on the `it-2004` dataset, since it is up to three orders of magnitude faster than the state-of-the-art algorithms. That is mainly because the ratio of updated edges to total edges in `it-2004` is larger than those in `dblp-coauthor`, `yahoo-song`, and `sx-stackoverflow`.

• **Time cost of different components in `HIT-Leiden`.** There are three key components, i.e., `inc-movement`, `inc-refine`, and `inc-aggregation`, in `HIT-Leiden`. We evaluate the proportion of time cost for each component and present the results in Figure 17. Note that some operations (e.g., `def-update` in `HIT-Leiden`) may not be included by the above three components, so we put them into the "Others" component. Notably, in `HIT-Leiden`, the refinement phase contributes minimally to the overall runtime. Besides, the combined proportion of time spent in its movement and aggregation phase is comparable to that of other algorithms. `Inc-movement`, `inc-refinement`, and `inc-aggregation` consistently outperform



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 12: Modularity values on dynamic graphs.



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 13: Modularity changes w.r.t. the number of update batches.

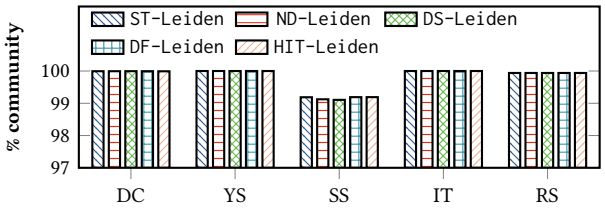
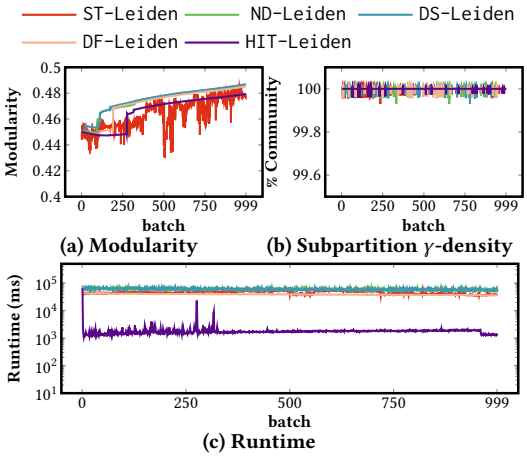


Figure 14: Percentage of subpartition γ-dense communities.



(a) Modularity (b) Subpartition γ-density

(c) Runtime

Figure 15: The effectiveness of HIT-Leiden over 999 update batches.

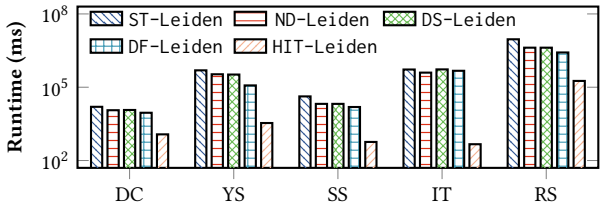


Figure 16: Efficiency of all Leiden algorithms on all datasets.

their counterparts in other algorithms across all datasets, achieving lower absolute runtime costs according to Figure 16.

• **Effect of b.** We vary the batch size $b \in \{10, 10^2, 10^3, 10^4, 10^5\}$ and report the efficiency in Figure 18. We see that HIT-Leiden is up to five orders of magnitude faster than other algorithms. Also, it exhibits a notable increase as b becomes smaller because it is a relatively bounded algorithm. In contrast, ND-Leiden, DS-Leiden, and DF-Leiden still need to process the entire graph when processing a new batch.

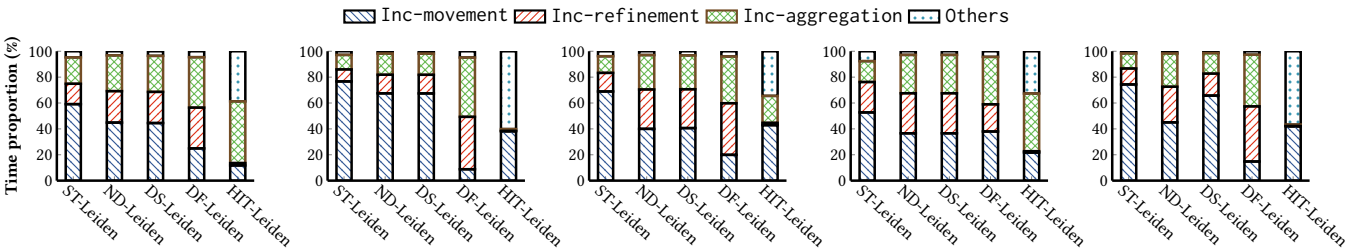
• **Effect of r.** Recall that after fixing the batch size b, we update the graph for r batches. Figure 19 shows the efficiency, where b is fixed as 1,000, but r ranges from 1 to 9. We observe that the incremental speedup is limited in the first few batches because $P = 10$ is small, and additional iterations may slightly improve the community membership. As a result, all the maintenance algorithms often require more time for the second batch to adjust the community structure. Once high-quality community structure is established, the speedup becomes significant. In addition, HIT-Leiden incurs a slightly higher runtime to record more information and construct the CC-index.

6.4 Two application studies

In this section, we apply HIT-Leiden to two real applications and analyze its empirical results.

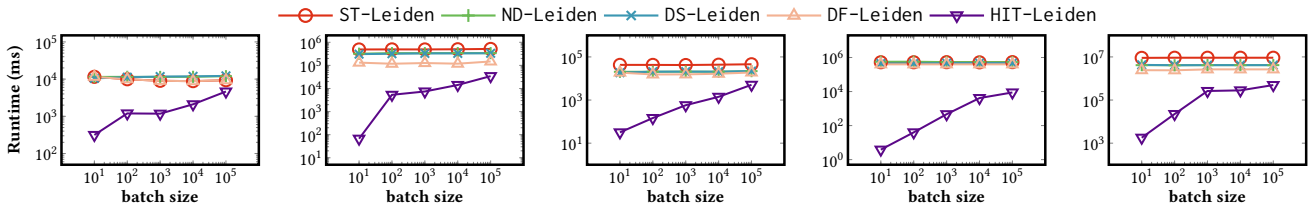
• **Graph-RAG.** We first briefly introduce the application of Graph-RAG. To augment the LLM generation for answering a question, people often retrieve relevant information from an external corpus. To facilitate retrieval, Graph-RAG builds an offline index: It first builds a graph for the corpus, then clusters the graph hierarchically using Leiden, and finally associates a summary for each community, which is generated by an LLM with some token cost. In practice, since the underlying corpus often changes, the communities and their summaries need to be updated as well. Our HIT-Leiden can not only dynamically update the communities efficiently, but also save the token cost since we only need to regenerate the summaries for the updated communities.

To experiment, we use the HotpotQA [87] dataset, which contains Wikipedia-based question-answer (QA) pairs. We randomly select 9,500 articles to build the initial graph, and insert 9 batches of new articles, each with 5 articles. The LLM we use is doubao-1.5-pro-32k. To support a dynamic corpus, we adapt the static Graph-RAG method by updating communities using ST-Leiden and HIT-Leiden, respectively. These two RAG methods are denoted



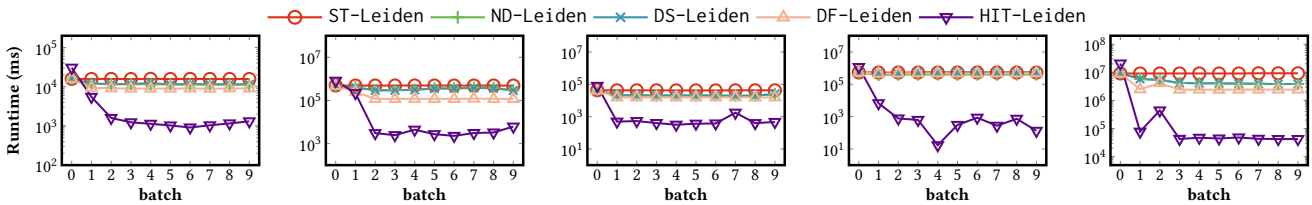
(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 17: Proportion of time cost of each component for the Leiden algorithms on all datasets.



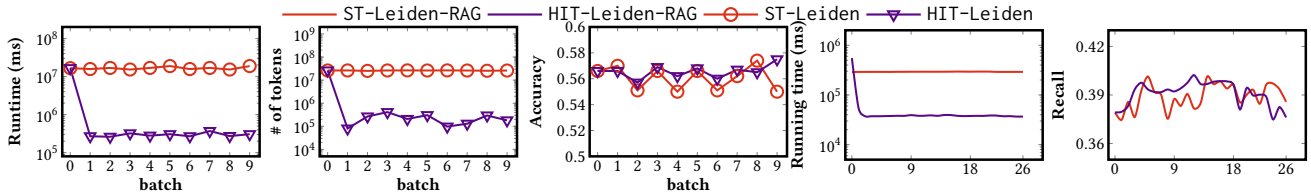
(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 18: Runtime on dynamic graphs.



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 19: Runtime w.r.t. the number of update batches.



(a) Runtime (b) Token cost (c) Accuracy (d) Runtime (e) Recall

Figure 20: Subfigures (a)–(c) compare ST-Leiden-RAG and HIT-Leiden-RAG over 9 update batches on RAG, and subfigures (d)–(e) compare ST-Leiden and HIT-Leiden over 26 update batches on fraud-ring discovery.

by ST-Leiden-RAG and HIT-Leiden-RAG, respectively. Note that ND-Leiden, DS-Leiden, and DF-Leiden are not fit to maintain the hierarchical communities of Graph-RAG since they lack hierarchical maintenance. We report their runtime, token cost, and accuracy in Figure 20(a)-(c). Clearly, HIT-Leiden-RAG is 56.1× faster than ST-Leiden-RAG. Moreover, it significantly reduces the summary token cost while preserving downstream QA accuracy, since its token cost is only 0.8% of the token cost of ST-Leiden-RAG. Hence, HIT-Leiden is effective for supporting Graph-RAG on a dynamic corpus.

• **Fraud-ring discovery.** We also experimented with a workflow of fraud-ring discovery at ByteDance. The input graph contains 58.3M vertices and 102.9M edges spanning 206 days, with daily updates (about 500,000 edge updates per batch). We construct a dynamic graph using a fixed 180-day sliding window: we initialize the graph with the first 180 days, and then update it day by day by adding edges from the new day while removing edges that fall outside the window. We evaluate the runtime ST-Leiden and HIT-Leiden, since other processes are black-box to us. Besides, we report the recall of finding high-risk individuals in Figure 20(d)-(e). Clearly, HIT-Leiden achieves 7.7× the speed of ST-Leiden with comparable recall.

7 CONCLUSIONS

In this paper, we develop an efficient algorithm for maintaining Leiden communities in a dynamic graph. We first theoretically analyze the boundedness of existing algorithms and how supervertex behaviors affect community membership under graph update. Building on these analyses, we further develop a relative boundedness algorithm, called HIT-Leiden, which consists of three key components, i.e., inc-movement, inc-refinement, and inc-aggregation. Extensive experiments on five real-world dynamic graphs show that HIT-Leiden not only preserves the properties of Leiden and achieves comparable modularity quality with Leiden, but also runs faster than state-of-the-art competitors. In future work, we will extend our algorithm to handle directed graphs and also evaluate it in a parallel environment.

REFERENCES

[1] 2026. Efficient Maintenance of Leiden Communities in Large Dynamic Graphs (full version). https://anonymous.4open.science/r/HIT_Leiden-Industry--2429/HIT-Leiden.pdf.

[2] Edo M Airoldi, David Blei, Stephen Fienberg, and Eric Xing. 2008. Mixed membership stochastic blockmodels. *Advances in neural information processing systems* 21 (2008).

[3] Arash A Amini, Aiyou Chen, Peter J Bickel, and Elizaveta Levina. 2013. Pseudo-likelihood methods for community detection in large sparse networks. (2013).

[4] Abdelouahab Amira, Abdelouahid Derhab, Elmouatez Billah Karbab, and Omar Nouali. 2023. A survey of malware analysis using community detection algorithms. *Comput. Surveys* 56, 2 (2023), 1–29.

[5] LN Fred Ana and Anil K Jain. 2003. Robust data clustering. In *2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2003. Proceedings.*, Vol. 2. IEEE, II–II.

[6] Thomas Aynaud and Jean-Loup Guillaume. 2010. Static community detection algorithms for evolving networks. In *8th international symposium on modeling and optimization in mobile, ad hoc, and wireless networks*. IEEE, 513–519.

[7] Thomas Aynaud and Jean-Loup Guillaume. 2011. Multi-step community detection and hierarchical time segmentation in evolving networks. In *Proceedings of the 5th SNA-KDD workshop*, Vol. 11.

[8] Vandana Bhatia and Rinkle Rani. 2018. Dfuzzy: a deep learning-based fuzzy clustering model for large graphs. *Knowledge and Information Systems* 57 (2018), 159–181.

[9] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.

[10] Stefan Boettcher and Allon G Percus. 2002. Optimization with extremal dynamics. *complexity* 8, 2 (2002), 57–62.

[11] Biao Cai, Yanpeng Wang, Lina Zeng, Yanmei Hu, and Hongjun Li. 2020. Edge classification based on convolutional neural networks for community detection in complex network. *Physica A: statistical mechanics and its applications* 556 (2020), 124826.

[12] Tanmoy Chakraborty, Ayushi Dalmia, Animesh Mukherjee, and Niloy Ganguly. 2017. Metrics for community analysis: A survey. *ACM Computing Surveys (CSUR)* 50, 4 (2017), 1–37.

[13] Qing Chen, Sven Helmer, Oded Lachish, and Michael Bohlen. 2022. Dynamic spanning trees for connectivity queries on fully-dynamic undirected graphs. (2022).

[14] Jiafeng Cheng, Qianqian Wang, Zhiqiang Tao, Deyan Xie, and Quanxue Gao. 2021. Multi-view attribute graph convolution networks for clustering. In *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence*. 2973–2979.

[15] Yun Chi, Xiaodan Song, Dengyong Zhou, Koji Hino, and Belle L Tseng. 2007. Evolutionary spectral clustering by incorporating temporal smoothness. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 153–162.

[16] Yun Chi, Xiaodan Song, Dengyong Zhou, Koji Hino, and Belle L Tseng. 2009. On evolutionary spectral clustering. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 3, 4 (2009), 1–30.

[17] Wen Haw Chong and Loo Nin Teow. 2013. An incremental batch technique for community detection. In *Proceedings of the 16th international conference on information fusion*. IEEE, 750–757.

[18] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 70, 6 (2004), 066111.

[19] Mário Cordeiro, Rui Portocarrero Sarmento, and Joao Gama. 2016. Dynamic community detection in evolving networks using locality modularity optimization. *Social Network Analysis and Mining* 6 (2016), 1–20.

[20] Ganqu Cui, Jie Zhou, Cheng Yang, and Zhiyuan Liu. 2020. Adaptive graph encoder for attributed graph embedding. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 976–985.

[21] Siemon C de Lange, Marcel A de Reus, and Martijn P van den Heuvel. 2014. The Laplacian spectrum of neural networks. *Frontiers in computational neuroscience* 7 (2014), 189.

[22] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, Vol. 96. 226–231.

[23] Shaohua Fan, Xiao Wang, Chuan Shi, Emiao Lu, Ken Lin, and Bai Wang. 2020. One2multi graph autoencoder for multi-view graph clustering. In *proceedings of the web conference 2020*. 3070–3076.

[24] Wenfei Fan, Chunming Hu, and Chao Tian. 2017. Incremental graph computations: Doable and undoable. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 155–169.

[25] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of the web conference 2020*. 2331–2341.

[26] László Gadár and János Abonyi. 2024. Explainable prediction of node labels in multilayer networks: a case study of turnover prediction in organizations. *Scientific Reports* 14, 1 (2024), 9036.

[27] Paul W Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. 1983. Stochastic blockmodels: First steps. *Social networks* 5, 2 (1983), 109–137.

[28] Jacob Holm, Kristian De Lichtenberg, and Mikkel Thorup. 2001. Poly-logarithmic deterministic fully-dynamic algorithms for connectivity, minimum spanning tree, 2-edge, and biconnectivity. *Journal of the ACM (JACM)* 48, 4 (2001), 723–760.

[29] Ruiqi Hu, Shirui Pan, Guodong Long, Qinghua Lu, Liming Zhu, and Jing Jiang. 2020. Going deep: Graph convolutional ladder-shape networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 2838–2845.

[30] Xiao Huang, Jundong Li, and Xia Hu. 2017. Accelerated attributed network embedding. In *Proceedings of the 2017 SIAM international conference on data mining*. SIAM, 633–641.

[31] Yuting Jia, Qinqin Zhang, Weinan Zhang, and Xinbing Wang. 2019. Community-gan: Community detection with generative adversarial nets. In *The world wide web conference*. 784–794.

[32] Baoyu Jing, Chanyoung Park, and Hanghang Tong. 2021. Hdmi: High-order deep multiplex infomax. In *Proceedings of the web conference 2021*. 2414–2424.

[33] Ravi Kannan, Santosh Vempala, and Adrian Vetta. 2004. On clusterings: Good, bad and spectral. *Journal of the ACM (JACM)* 51, 3 (2004), 497–515.

[34] Brian Karrer and Mark EJ Newman. 2011. Stochastic blockmodels and community structure in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 83, 1 (2011), 016107.

[35] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. 1983. Optimization by simulated annealing. *science* 220, 4598 (1983), 671–680.

[36] Sadamori Kojaku, Giacomo Livan, and Naoki Masuda. 2021. Detecting anomalous citation groups in journal networks. *Scientific Reports* 11, 1 (2021), 14524.

[37] Andrea Lancichinetti and Santo Fortunato. 2009. Community detection algorithms: a comparative analysis. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 80, 5 (2009), 056117.

[38] Ron Levie, Federico Monti, Xavier Bresson, and Michael M Bronstein. 2018. Cayleynets: Graph convolutional neural networks with complex rational spectral filters. *IEEE Transactions on Signal Processing* 67, 1 (2018), 97–109.

[39] Bentian Li, Dechang Pi, Yunxia Lin, and Lin Cui. 2021. DNC: A deep neural network-based clustering-oriented network embedding algorithm. *Journal of Network and Computer Applications* 173 (2021), 102854.

[40] Zhangtao Li and Jing Liu. 2016. A multi-agent genetic algorithm for community detection in complex networks. *Physica A: Statistical Mechanics and its Applications* 449 (2016), 336–347.

[41] Xujian Liang and Zhaoquan Gu. 2025. Fast think-on-graph: Wider, deeper and faster reasoning of large language model on knowledge graph. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24558–24566.

[42] Yu-Ru Lin, Yun Chi, Shenghuo Zhu, Hari Sundaram, and Belle L Tseng. 2008. Facetnet: a framework for analyzing communities and their evolutions in dynamic networks. In *Proceedings of the 17th international conference on World Wide Web*. 685–694.

[43] Fanzhen Liu, Jia Wu, Chuan Zhou, and Jian Yang. 2019. Evolutionary community detection in dynamic social networks. In *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–7.

[44] Yanbei Liu, Xiao Wang, Shu Wu, and Zhitao Xiao. 2020. Independence promoted graph disentangled networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 4916–4923.

[45] Linhao Luo, Yixiang Fang, Xin Cao, Xiaofeng Zhang, and Wenjie Zhang. 2021. Detecting communities from heterogeneous graphs: A context path-based graph neural network model. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 1170–1180.

[46] Aaron F McDaid, Derek Greene, and Neil Hurley. 2011. Normalized mutual information to evaluate overlapping community finding algorithms. *arXiv preprint arXiv:1110.2515* (2011).

[47] Xiangfeng Meng, Yunhai Tong, Xinhai Liu, Shuai Zhao, Xianglin Yang, and Shaohua Tan. 2016. A novel dynamic community detection algorithm based on modularity optimization. In *2016 7th IEEE international conference on software engineering and service science (ICSESS)*. IEEE, 72–75.

[48] Microsoft. 2025. GraphRAG: A Structured, Hierarchical Approach to Retrieval Augmented Generation. https://microsoft.github.io/graphrag/. Accessed: 2025-03-31.

[49] Jennifer Neumann and Peter M Fischer. [n.d.]. Shape-Aware, Scale-Agnostic Representation of Dynamic DAGs. *Proceedings of the VLDB Endowment. ISSN* 2150 ([n. d.]), 8097.

[50] Mark EJ Newman. 2004. Fast algorithm for detecting community structure in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 69, 6 (2004), 066133.

[51] Mark EJ Newman. 2006. Finding community structure in networks using the eigenvectors of matrices. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 74, 3 (2006), 036104.

[52] Mark EJ Newman. 2006. Modularity and community structure in networks. *Proceedings of the national academy of sciences* 103, 23 (2006), 8577–8582.

[53] Mark EJ Newman. 2013. Spectral methods for community detection and graph partitioning. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 88, 4 (2013), 042822.

[54] Nam P Nguyen, Thang N Dinh, Sindhura Tokala, and My T Thai. 2011. Overlapping communities in dynamic networks: their detection and mobile applications. In *Proceedings of the 17th annual international conference on Mobile computing and networking*. 85–96.

[55] Nam P Nguyen, Thang N Dinh, Ying Xuan, and My T Thai. 2011. Adaptive algorithms for detecting community structure in dynamic social networks. In *2011 Proceedings IEEE INFOCOM*. IEEE, 2282–2290.

[56] Alexandru Oarga, Matthew Hart, Andres M Bran, Magdalena Lederbauer, and Philippe Schwaller. 2024. Scientific knowledge graph and ontology generation using open large language models. In *AI for Accelerated Materials Design-NeurIPS 2024*.

[57] Shashank Pandit, Duen Horng Chau, Samuel Wang, and Christos Faloutsos. 2007. Netprobe: a fast and scalable system for fraud detection in online auction networks. In *Proceedings of the 16th international conference on World Wide Web*. 201–210.

[58] Songtao Peng, Jiaqi Nie, Xincheng Shu, Zhongyuan Ruan, Lei Wang, Yunxuan Sheng, and Qi Xuan. 2022. A multi-view framework for BGP anomaly detection via graph attention network. *Computer Networks* 214 (2022), 109129.

[59] Ganesan Ramalingam and Thomas Reps. 1996. On the computational complexity of dynamic graph problems. *Theoretical Computer Science* 158, 1-2 (1996), 233–277.

[60] Boyu Ruan, Junhao Gan, Hao Wu, and Anthony Wirth. 2021. Dynamic structural clustering on graphs. In *Proceedings of the 2021 International Conference on Management of Data*. 1491–1503.

[61] Subhajit Sahu. 2024. DF Louvain: Fast Incrementally Expanding Approach for Community Detection on Dynamic Graphs. *arXiv preprint arXiv:2404.19634* (2024).

[62] Subhajit Sahu. 2024. A Starting Point for Dynamic Community Detection with Leiden Algorithm. *arXiv preprint arXiv:2405.11658* (2024).

[63] Subhajit Sahu, Kishore Kothapalli, and Dip Sankar Banerjee. 2024. Fast Leiden Algorithm for Community Detection in Shared Memory Setting. In *Proceedings of the 53rd International Conference on Parallel Processing*. 11–20.

[64] Arindam Sarkar, Nikhil Mehta, and Piyush Rai. 2020. Graph representation learning via ladder gamma variational autoencoders. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 5604–5611.

[65] Jiaxing Shang, Lianchen Liu, Xin Li, Feng Xie, and Cheng Wu. 2016. Targeted revision: A learning-based approach for incremental community detection in dynamic networks. *Physica A: Statistical Mechanics and its Applications* 443 (2016), 70–85.

[66] Jiaxing Shang, Lianchen Liu, Feng Xie, Zhen Chen, Jiajia Miao, Xuelin Fang, and Cheng Wu. 2014. A real-time detecting algorithm for tracking community structure of dynamic networks. *arXiv preprint arXiv:1407.2683* (2014).

[67] Oleksandr Shchur and Stephan Günnemann. 2019. Overlapping community detection with graph neural networks. *arXiv preprint arXiv:1909.12201* (2019).

[68] Stanislav Sobolevsky, Riccardo Campari, Alexander Belyi, and Carlo Ratti. 2014. General optimization technique for high-quality community detection in complex networks. *Physical Review E* 90, 1 (2014), 012811.

[69] Xing Su, Shan Xue, Fanzhen Liu, Jia Wu, Jian Yang, Chuan Zhou, Wenbin Hu, Cecile Paris, Surya Nepal, Di Jin, et al. 2022. A comprehensive survey on community detection with deep learning. *IEEE transactions on neural networks and learning systems* 35, 4 (2022), 4682–4702.

[70] Wenwen Sun, Zhicheng Pan, Zirui Hu, Yu Liu, Chengcheng Yang, Rong Zhang, and Xuan Zhou. 2025. Rabbit: Retrieval-augmented generation enables better automatic database knob tuning. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3807–3820.

[71] Tencent. 2019. *Tencent Graph Computing (TGraph) Officially Open Sourced High-Performance Graph Computing Framework: Plato*. Accessed: 2025-04-17.

[72] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific reports* 9, 1 (2019), 1–12.

[73] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[74] Lewen Wang, Haozhe Zhao, Cunguang Feng, Weiqing Liu, Congrui Huang, Marco Santoni, Manuel Cristofaro, Paola Jafrancesco, and Jiang Bian. 2023. Removing camouflage and revealing collusion: Leveraging gang-crime pattern in fraudster detection. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 5104–5115.

[75] Shu Wang, Yixiang Fang, and Wensheng Luo. 2025. Searching and Detecting Structurally Similar Communities in Large Heterogeneous Information Networks. *Proceedings of the VLDB Endowment* 18, 5 (2025), 1425–1438.

[76] Xiao Wang, Nian Liu, Hui Han, and Chuan Shi. 2021. Self-supervised heterogeneous graph neural network with co-contrastive learning. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 1726–1736.

[77] Wei Xia, Qianqian Wang, Quanxue Gao, Xiangdong Zhang, and Xinbo Gao. 2021. Self-supervised graph convolutional network for multi-view clustering. *IEEE Transactions on Multimedia* 24 (2021), 3182–3192.

[78] Jierui Xie, Mingming Chen, and Boleslaw K Szymanski. 2013. LabelrankT: Incremental community detection in dynamic networks via label propagation. In *Proceedings of the workshop on dynamic networks management and mining*. 25–32.

[79] Jierui Xie and Boleslaw K Szymanski. 2013. Labelrank: A stabilized label propagation algorithm for community detection in networks. In *2013 IEEE 2nd Network Science Workshop (NSW)*. IEEE, 138–143.

[80] Jierui Xie, Boleslaw K Szymanski, and Xiaoming Liu. 2011. Slpa: Uncovering overlapping communities in social networks via a speaker-listener interaction dynamic process. In *2011 ieee 11th international conference on data mining workshops*. IEEE, 344–349.

[81] Yu Xie, Maoguo Gong, Shanfeng Wang, and Bin Yu. 2018. Community discovery in networks with deep sparse filtering. *Pattern Recognition* 81 (2018), 50–59.

[82] Lantian Xu, Dong Wen, Lu Qin, Ronghua Li, Ying Zhang, and Xuemin Lin. 2024. Constant-time Connectivity Querying in Dynamic Graphs. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–23.

[83] Rongbin Xu, Yan Che, Xinmei Wang, Jianxiong Hu, and Ying Xie. 2020. Stacked autoencoder-based community detection method via an ensemble clustering framework. *Information sciences* 526 (2020), 151–165.

[84] Xiaowei Xu, Nurcan Yuruk, Zhidan Feng, and Thomas AJ Schweiger. 2007. Scan: a structural clustering algorithm for networks. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 824–833.

[85] Liang Yang, Xiaochun Cao, Dongxiao He, Chuan Wang, Xiao Wang, and Weixiong Zhang. 2016. Modularity based community detection with deep learning.. In *IJCAI*, Vol. 16. 2252–2258.

[86] Zhao Yang, René Algesheimer, and Claudio J Tessone. 2016. A comparative analysis of community detection algorithms on artificial networks. *Scientific reports* 6, 1 (2016), 30750.

[87] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600* (2018).

[88] Quanzeng You, Hailin Jin, Zhaowen Wang, Chen Fang, and Jiebo Luo. 2016. Image captioning with semantic attention. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4651–4659.

[89] Neda Zarayeneh and Ananth Kalyanaraman. 2021. Delta-screening: a fast and efficient technique to update communities in dynamic graphs. *IEEE transactions on network science and engineering* 8, 2 (2021), 1614–1629.

[90] Fangyuan Zhang and Sibo Wang. 2022. Effective indexing for dynamic structural graph clustering. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2908–2920.

[91] Tianqi Zhang, Yun Xiong, Jiawei Zhang, Yao Zhang, Yizhu Jiao, and Yangyong Zhu. 2020. CommDGI: community detection oriented deep graph infomax. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1843–1852.

[92] Xiaotong Zhang, Han Liu, Xiao-Ming Wu, Xianchao Zhang, and Xinyue Liu. 2021. Spectral embedding network for attributed graph clustering. *Neural Networks* 142 (2021), 388–396.

[93] Yao Zhang, Yun Xiong, Yun Ye, Tengfei Liu, Weiqiang Wang, Yangyong Zhu, and Philip S Yu. 2020. SEAL: Learning heuristics for community detection with generative adversarial networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 1103–1113.

[94] Han Zhao, Xu Yang, Zhenru Wang, Erkun Yang, and Cheng Deng. 2021. Graph debiased contrastive learning with joint representation clustering.. In *IJCAI*. 3434–3440.

[95] Yingli Zhou, Qingshuo Guo, Yi Yang, Yixiang Fang, Chenhao Ma, and Laks Lakshmanan. 2024. In-depth Analysis of Densest Subgraph Discovery in a Unified Framework. *arXiv preprint arXiv:2406.04738* (2024).

[96] Di Zhuang, J Morris Chang, and Mingchen Li. 2019. DynaMo: Dynamic community detection by incrementally maximizing modularity. *IEEE Transactions on Knowledge and Data Engineering* 33, 5 (2019), 1934–1945.

APPENDIX

A DISCUSSION

In this appendix, we explain how Observations 2–4 are derived, and introduce several lemmas along the way. For the sake of a coherent presentation, the proofs of these lemmas are given in Appendix B.

(1) Edge deletion. We consider the deletions of both intra-sub-community edges and cross-sub-community edges:

Lemma 2. *Given an intra-sub-community edge deletion $(v_i, v_j, -\alpha)$, assume v_j is before v_i in the γ-order of the sub-community. The effects of the edge deletion can be described by the following four cases:*

(1) *v_i could be removed from its sub-community only if $\alpha > \frac{2m \cdot w(v_i,U_i) - \gamma \cdot d(v_i) \cdot d(U_i)}{4m + 2w(v_i,U_i)}$;*
(2) *v_j could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{2w(v_j,U_j)}$;*
(3) *For any $v_k \in S_i$ $(k \neq i, j)$, it could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k,U_k)}$;*
(4) *For any $v_l \notin S_i$, it should be removed from its sub-community if and only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l))}{2w(v_l,U_l)}$.*

Lemma 3. *Given a cross-sub-community edge deletion $(v_i, v_j, -\alpha)$, the effects of the edge deletion can be described by the following four cases:*

(1) *v_i could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{2w(v_i,U_i)}$;*
(2) *v_j holds similar behavior with v_i;*
(3) *For any $v_k \in S_i \cup S_j$ $(k \neq i, j)$, it could be removed its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k,U_k)}$;*
(4) *For any $v_l \notin S_i \cup S_j$, it could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l,U_l)}$.*

(2) Edge insertion. We consider the insertion of edges containing the insertions of both intra-sub-community edges and cross-sub-community edges:

Lemma 4. *Given an edge insertion (v_i, v_j, α), the effects of the edge insertion can be described by the following four cases:*

(1) *v_i could be removed from its sub-community only if $\alpha > \frac{4}{\gamma} m - d(I_i)$ or $\alpha > \frac{2w(v_i,U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$;*
(2) *v_j could be removed from its sub-community, only if $\alpha > \frac{2w(v_j,U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$;*
(3) *For any $v_k \in S_i \cup S_j$ $(k \neq i, j)$, it could be removed from its sub-community, only if $\alpha > \frac{w(v_k,U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2} d(U_k)$;*
(4) *For any $v_l \notin S_i \cup S_j$, it is unaffected.*

Observation 6. *In the refinement phase of Leiden algorithms, each vertex v is likely to be merged into the sub-community (intermediate subsequence U), offering more edge weights $w(v, U)$ and smaller degrees $d(U)$. Therefore, the differences of the values of $d(v)$, $w(v, U)$, and $d(U)$ are very small when the traversal order of vertices to be merged into sub-communities is in ascending order of vertex degree.*

By the above observation, α is unlikely to satisfy the conditions in cases (2)-(4) of Lemma 2, all the cases of Lemma 3, and the conditions in cases (1)-(3) of Lemma 4 when $\alpha \ll m$ (which is often true as stated in Assumption 1). As a result, when designing the maintenance algorithm, we only need to consider the effect of intra-sub-community edge deletions on v_i, which cannot be ignored.

B PROOF OF LEMMAS

B.1 Proof of Lemma 1

Proof. Assuming v changes its community from C_i to C, there are three cases:

(1) For each neighbor v_i in C_i, the edge (v, v_i) is a *deleted intra-community* edge and an inserted cross-community edge;
(2) For each neighbor v_j in C, the edge (v, v_j) is a deleted cross-community edge and an inserted intra-community edge;
(3) For each other neighbor v_k, edge (v, v_k) is a deleted cross-community edge and an *inserted cross-community* edge.

Since only the first and third cases meet the conditions in Observation 1, all the neighbors of v that are not in C are likely to change their communities. □

B.2 Proof of Lemma 2

Proof. We analyze the modularity gain $\Delta M(v \to \emptyset, \gamma)$ for any vertex v, which denotes the modularity gain of moving v from the intermediate subsequence I to $\emptyset$, whose calculation follows the same formula as the standard modularity gain.

According to Definition 7, if $\Delta M(v \to \emptyset, \gamma) > 0$, the intermediate subsequence I could not be γ-connected and v has to leave I. It is different from maintaining vertex optimality (mentioned in Definition 5): If there exists a community C' such that the modularity gain of moving v from its community C to C' is positive, v is not locally optimized and has to be removed from C.

Case 1: v_i is inserted into S after v_j, i.e., $v_j \in I_i$. The old modularity gain $M_{old}(v_i \to \emptyset, \gamma) < 0$ before deletion is:

$$M_{old}(v_i \to \emptyset, \gamma) = -\frac{w(v_i, U_i)}{2m} + \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{4m^2} \leq 0. \tag{2}$$

Where $U_i = I_i \setminus \{v_i\}$. We multiply right side of Equation (2) by $4m^2$ and obtain $X_{(2)}$:

$$X_{(2)} = -2m \cdot w(v_i, U_i) + \gamma \cdot d(v_i) \cdot d(U_i) \leq 0 \tag{3}$$

After the deletion, the new modularity gain $M_{new}(v_i \to \emptyset, \gamma)$ formulates:

$$\begin{aligned} \Delta M_{new}(v_i \to \emptyset, \gamma) = &-\frac{w(v_i, U_i) - 2\alpha}{2(m - \alpha)} \\ &+ \frac{\gamma \cdot (d(v_i) - \alpha) \cdot (d(U_i) - \alpha)}{4(m - \alpha)^2}. \end{aligned} \tag{4}$$

We multiply right side of Equation (4) by $4(m - \alpha)^2$ and obtain $Y_{(4)}$:

$$\begin{aligned} Y_{(4)} &= -2(m - \alpha) \cdot (w(v_i, U_i) - 2\alpha) \\ &\quad + \gamma \cdot (d(v_i) - \alpha) \cdot (d(U_i) - \alpha) \\ &= X_{(2)} + \alpha \cdot (4m + 2w(v_i, U_i) - 4a - \gamma \cdot (d(I_i) - \alpha)) \\ &< X_{(2)} + \alpha \cdot (4m + 2w(v_i, U_i)) \end{aligned} \tag{5}$$

If $X_{(2)} + \alpha \cdot (4m + 2w(v_i, U_i)) > 0$, $\Delta M_{new}(v_i \to \emptyset, \gamma)$ could be positive; Otherwise, $\Delta M_{new}(v_i \to \emptyset, \gamma)$ must be non-positive. Therefore, v_i could be removed from its sub-community only if $\alpha > \frac{2m \cdot w(v_i,U_i) - \gamma \cdot d(v_i) \cdot d(U_i)}{4m + 2w(v_i,U_i)}$.

Case 2: v_j is inserted into S before v_i. In this case, we have $v_j \in I_j, v_i \notin I_j$, and the edge deletion does not affect intra-edges within U_j. The old modularity gain $M_{old}(v_i \to \emptyset, \gamma) < 0$ before deletion is:

$$M_{old}(v_j \to \emptyset, \gamma) = -\frac{w(v_j, U_j)}{2m} + \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{4m^2}. \tag{6}$$

We multiply right side of Equation (2) by $4m^2$ and obtain $X_{(2)}$:

$$X_{(6)} = -2m \cdot w(v_j, U_j) + \gamma \cdot d(v_j) \cdot d(U_j) < 0 \tag{7}$$

The new modularity gain after the edge deletion becomes:

$$\begin{aligned} \Delta M_{new}(v_j \to \emptyset, \gamma) = &-\frac{w(v_j, U_j)}{2(m-\alpha)} \\ &+ \frac{\gamma \cdot (d(v_j) - \alpha) \cdot d(U_j)}{4(m-\alpha)^2} \end{aligned} \tag{8}$$

We multiply right side of Equation (8) by $4(m-\alpha)^2$ and obtain $Y_{(8)}$:

$$\begin{aligned} Y_{(8)} &= -2(m-\alpha) \cdot w(v_j, U_j) + \gamma \cdot (d(v_j) - \alpha) \cdot d(U_j) \\ &= X_{(6)} + 2\alpha \cdot \big(w(v_j, U_j) - \gamma \cdot d(U_j)\big) \\ &< X_{(6)} + 2\alpha \cdot w(v_j, U_j) \end{aligned} \tag{9}$$

Hence, v_j could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{2w(v_j, U_j)}$.

Generalization to other vertices. Consider other vertices v_k and v_l such that $v_k \in S_i$, $k \neq i, j$ and $v_l \notin S_i$. The old modularity gains $M_{old}(v_k \to \emptyset, \gamma) < 0$ and $M_{old}(v_l \to \emptyset, \gamma) < 0$ before deletion are:

$$M_{old}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2m} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{4m^2}. \tag{10}$$

$$M_{old}(v_l \to \emptyset, \gamma) = -\frac{w(v_l, U_l)}{2m} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4m^2}. \tag{11}$$

We multiply right side of Equation (10) and (11) by $4m^2$ respectively to obtain $X_{(10)}$ and $X_{(11)}$:

$$X_{(10)} = -2m \cdot w(v_k, U_k) + \gamma \cdot d(v_k) \cdot d(U_k) \leq 0 \tag{12}$$

$$X_{(11)} = -2m \cdot w(v_l, U_l) + \gamma \cdot d(v_l) \cdot d(U_l) \leq 0 \tag{13}$$

After the edge deletion, their new modularity gains are satisfied:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) \leq -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{4(m-\alpha)^2}. \tag{14}$$

$$\Delta M_{new}(v_l \to \emptyset, \gamma) = -\frac{w(v_l, U_l)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4(m-\alpha)^2}. \tag{15}$$

v_k could be merged before v_i and v_j, between v_i and v_j, as well as after v_i and v_j. $\Delta M_{new}(v_k \to \emptyset, \gamma)$ can be formulated as follows:

(1) v_k is merged before v_i and v_j:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{4(m-\alpha)^2}; \tag{16}$$

(2)v_k is merged between v_i and v_j:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) - \alpha)}{4(m-\alpha)^2}; \tag{17}$$

(3) v_k is merged after v_i and v_j:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) - 2\alpha)}{4(m-\alpha)^2}. \tag{18}$$

Therefore, the equivalent of Equation (14) holds if and only if v_k is merged before v_i and v_j. Then, We multiply right side of Equation (14) and (15) by $4(m-\alpha)^2$ respectively and obtain $Y_{(14)}$ and $Y_{(15)}$:

$$\begin{aligned} Y_{(14)} &= -2(m-\alpha) \cdot w(v_k, U_k) \\ &\quad + \gamma \cdot d(v_k) \cdot d(U_k) \\ &= X_{12} + 2\alpha \cdot w(v_k, U_k), \end{aligned} \tag{19}$$

$$Y_{(15)} = X_{13} + 2\alpha \cdot w(v_l, U_l), \tag{20}$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k, U_k)}$, v_k could be removed from its sub-community; v_l should be removed from its sub-community if and only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l, U_l)}$.

□

B.3 Proof of Lemma 3

Proof. We adopt the same notations as in the proof of Lemma 2, with the exception that v_k now denotes a vertex residing in the same sub-community as either v_i or v_j. Based on this setup, the modularity gain after the edge deletion is shown as follows.

Case 1: Consider the endpoint v_i:

$$\begin{aligned} \Delta M_{new}(v_i \to \emptyset, \gamma) = &-\frac{w(v_i, U_i)}{2(m-\alpha)} \\ &+ \frac{\gamma \cdot (d(v_i) - \alpha) \cdot d(U_i)}{4(m-\alpha)^2}. \end{aligned} \tag{21}$$

We multiply right side of Equation (21) by $4(m-\alpha)^2$ and obtain $Y_{(21)}$:

$$\begin{aligned} Y_{(21)} &= -2(m-\alpha) \cdot w(v_i, U_i) \\ &\quad + \gamma \cdot (d(v_i) - \alpha) \cdot d(U_i) \\ &= X_{(2)} + \alpha \cdot (2w(v_i, U_i) - \gamma \cdot d(U_i)) \\ &< X_{(2)} + \alpha \cdot 2w(v_i, U_i) \end{aligned} \tag{22}$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{2w(v_i, U_i)}$, v_i could be removed from its sub-community. v_j holds similar behavior.

Case 2: Consider the vertex $v_k \in S_i \cup S_j$, $k \neq i, j$:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) \leq -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{4(m-\alpha)^2}. \tag{23}$$

For Equation (23), v_k could be merged before v_i or v_j, as well as after v_i or v_j. Its equivalent holds if and only if v_k is merged before v_i or v_j. We multiply right side of Equation (23) by $4(m-\alpha)^2$ and obtain $Y_{(23)}$:

$$\begin{aligned} Y_{(23)} &= -2(m-\alpha) \cdot w(v_k, U_k) + \gamma \cdot d(v_k) \cdot d(U_k) \\ &= X_{(10)} + 2\alpha \cdot w(v_k, U_k) \end{aligned} \tag{24}$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k, U_k)}$, v_k could be removed from its sub-community.

Case 3: Consider the vertex $v_l \notin S_i \cup S_j$:

$$\Delta M_{new}(v_l \to \emptyset, \gamma) = -\frac{w(v_l, U_l)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4(m-\alpha)^2}. \tag{25}$$

Similar to **Case 2**, if and only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l, U_l)}$, v_l should be removed from its sub-community.

□

B.4 Proof of Lemma 4

Proof. First, we analyze the **insertion of intra-sub-community edges**. We adopt the same notations as in the proof of Lemma 2. Based on this setup, the modularity gain after the edge insertion is shown as follows.

Case 1: Consider the endpoint v_i, which is the latter merged endpoint:

$$\begin{aligned}\Delta M_{new}(v_i \to \emptyset, \gamma) = &-\frac{w(v_i, U_i) + 2\alpha}{2(m+\alpha)} \\ &+ \frac{\gamma \cdot (d(v_i) + \alpha) \cdot (d(U_i) + \alpha)}{4(m+\alpha)^2}.\end{aligned} \tag{26}$$

We multiply right side of Equation (26) by $4(m+\alpha)^2$ and obtain $Y_{(26)}$:

$$\begin{aligned}Y_{(26)} &= -2(m+\alpha)(w(v_i, U_i) + 2\alpha) \\ &\quad + \gamma \cdot (d(v_i) + \alpha) \cdot (d(U_i) + \alpha) \\ &= X_{(2)} + \alpha \cdot \big(\gamma \cdot (d(I_i) + \alpha) - 2w(v_i, U_i) - 4\alpha - 4m\big) \\ &< X_{(2)} + \alpha \cdot \big(\gamma \cdot (d(I_i) + \alpha) - 4m\big)\end{aligned} \tag{27}$$

Obviously, only if $\gamma \cdot (d(I_i) + \alpha) - 4m > 0$, i.e., $\alpha > \frac{4}{\gamma} m - d(I_i)$, $Y_{(26)}$ could be positive.

Case 2: Consider the endpoint v_j, which is the former merged endpoint:

$$\begin{aligned}\Delta M_{new}(v_j \to \emptyset, \gamma) = &-\frac{w(v_j, U_i)}{2(m+\alpha)} \\ &+ \frac{\gamma \cdot (d(v_j) + \alpha) \cdot d(U_i)}{4(m+\alpha)^2}.\end{aligned} \tag{28}$$

We multiply right side of Equation (28) by $4(m+\alpha)^2$ and obtain $Y_{(28)}$:

$$\begin{aligned}Y_{(28)} &= -2(m+\alpha) \cdot w(v_j, U_j) \\ &\quad + \gamma \cdot (d(v_j) + \alpha) \cdot d(U_j) \\ &= X_{(6)} + \alpha \cdot (\gamma \cdot d(U_j) - w(v_j, U_j)) \\ &< X_{(6)} + \alpha \cdot \gamma \cdot d(U_j)\end{aligned} \tag{29}$$

Only if $\alpha > \frac{2w(v_j, U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$, v_j could be removed from its sub-community.

Case 3: Consider other vertex $v_k \in S_i$, $k \neq i, j$:

$$\begin{aligned}\Delta M_{new}(v_k \to \emptyset, \gamma) \le &-\frac{w(v_k, U_k)}{2(m+\alpha)} \\ &+ \frac{\gamma \cdot d(v_k) \cdot (d(U_k) + 2\alpha)}{4(m+\alpha)^2}.\end{aligned} \tag{30}$$

The equivalent of Equation (30) holds if and only if v_k is merged after v_i and v_j. We multiply right side of Equation (30) by $4(m+\alpha)^2$ and obtain $Y_{(30)}$:

$$\begin{aligned}Y_{(30)} &= -2(m+\alpha) \cdot w(v_k, U_k) \\ &\quad + \gamma \cdot d(v_k) \cdot (d(U_k) + 2\alpha) \\ &= X_{(10)} + \alpha \cdot \big(2\gamma \cdot d(v_k) - 2w(v_k, U_k)\big) \\ &< X_{(10)} + 2\alpha \cdot \gamma \cdot d(v_k)\end{aligned} \tag{31}$$

Only if $\alpha > \frac{w(v_k, U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2} d(U_k)$, v_k could be removed from its sub-community.

Case 4: Consider other vertex $v_l \notin S_i$:

$$\begin{aligned}\Delta M_{new}(v_l \to \emptyset, \gamma) \le &-\frac{w(v_l, U_l)}{2(m+\alpha)} \\ &+ \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4(m+\alpha)^2}.\end{aligned} \tag{32}$$

Equation (32) holds if and only if v_j is merged after v_i and v_j. We multiply right side of Equation (32) by $4(m+\alpha)^2$ and obtain $Y_{(30)}$:

$$\begin{aligned}Y_{(32)} &= -2(m+\alpha) \cdot w(v_l, U_l) \\ &\quad + \gamma \cdot d(v_l) \cdot d(U_l) \\ &= X_{(11)} - 2\alpha \cdot w(v_l, U_l) < 0\end{aligned} \tag{33}$$

v_l is not affected by the intra-sub-community insertion.

Now, we consider the **insertion of cross-sub-community edges**. We adopt the same notations as in the proof of Lemma 3. Based on this setup, the modularity gain after the edge insertion is shown as follows.

Case 5: Consider the endpoint v_i:

$$\begin{aligned}\Delta M_{new}(v_i \to \emptyset, \gamma) = &-\frac{w(v_i, U_i)}{2(m+\alpha)} \\ &+ \frac{\gamma \cdot (d(v_i) + \alpha) \cdot d(U_i)}{4(m+\alpha)^2}.\end{aligned} \tag{34}$$

We multiply right side of Equation (34) by $4(m+\alpha)^2$ and obtain $Y_{(34)}$:

$$\begin{aligned}Y_{(34)} &= -2(m+\alpha) \cdot w(v_i, U_i) \\ &\quad + \gamma \cdot (d(v_i) + \alpha) \cdot d(U_i) \\ &= X_{(2)} + \alpha \cdot \big(\gamma \cdot d(U_i) - 2w(v_i, U_i)\big) \\ &< X_{(2)} + \alpha \cdot \gamma \cdot d(U_i)\end{aligned} \tag{35}$$

Only if $\alpha > \frac{2w(v_i, U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$, v_i could be removed from its sub-community. v_j is the same.

Case 6: Consider other vertex $v_k \in S_i \cup S_j$, $k \neq i, j$:

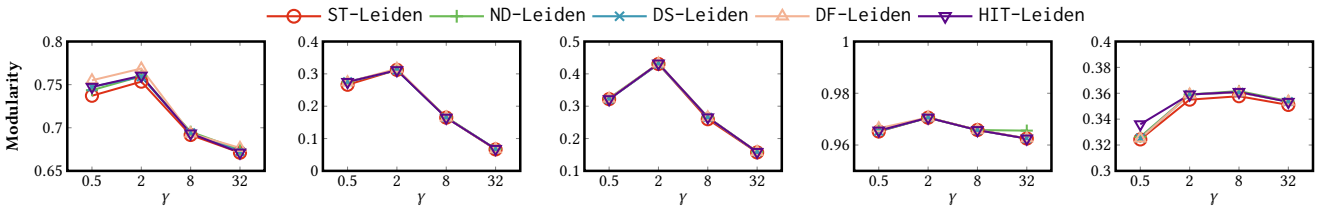
$$\begin{aligned}\Delta M_{new}(v_k \to \emptyset, \gamma) \le &-\frac{w(v_k, U_k)}{2(m+\alpha)} \\ &+ \frac{\gamma \cdot d(v_k) \cdot (d(U_k) + \alpha)}{4(m+\alpha)^2}.\end{aligned} \tag{36}$$

The equivalent of Equation (36) holds if and only if v_k is merged after v_i or v_j. We multiply right side of Equation (36) by $4(m+\alpha)^2$ and obtain $Y_{(36)}$:

$$\begin{aligned}Y_{(36)} &= -2(m+\alpha) \cdot w(v_i, U_i) \\ &\quad + \gamma \cdot d(v_i) \cdot (d(U_i) + \alpha) \\ &= X_{(2)} + \alpha \cdot \big(\gamma \cdot d(v_i) - 2w(v_i, U_i)\big) \\ &< X_{(2)} + \alpha \cdot \gamma \cdot d(v_i) \\ &< X_{(2)} + 2\alpha \cdot \gamma \cdot d(v_i)\end{aligned} \tag{37}$$

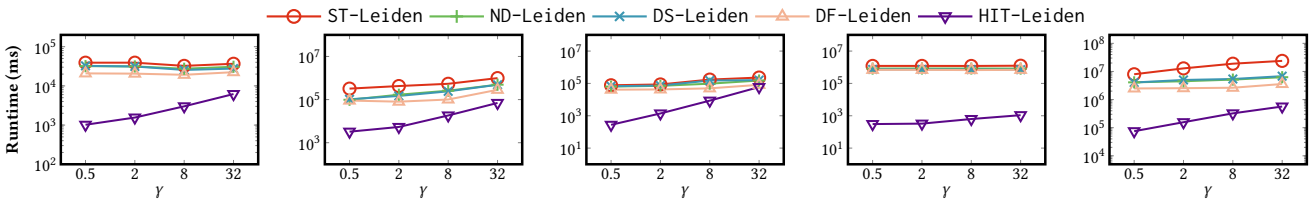
v_k could be removed from its sub-community only if $\alpha > \frac{w(v_k, U_k)}{\gamma d(v_k)} \cdot m - \frac{1}{2} d(U_k)$.

Case 7: Consider other vertex $v_l \notin S_i$:



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 21: Modularity values w.r.t. γ.



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 22: Runtime w.r.t. γ.

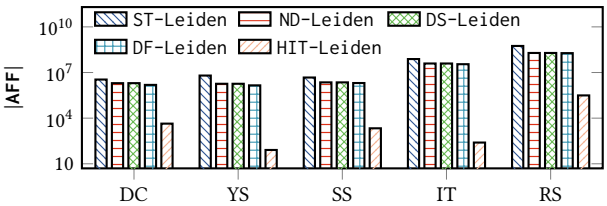


Figure 23: Effect of |AFF| on all datasets.

$$\Delta M_{new}(v_l \to \emptyset, \gamma) \leq -\frac{w(v_l, U_l)}{2(m+\alpha)} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4(m+\alpha)^2}. \tag{38}$$

Equation (38) holds if and only if v_j is merged after v_i and v_j. We multiply right side of Equation (38) by $4(m+\alpha)^2$ and obtain $Y_{(36)}$:

$$\begin{aligned} Y_{(38)} &= -2(m+\alpha) \cdot w(v_l, U_l) \\ &\quad + \gamma \cdot d(v_l) \cdot d(U_l) \\ &= X_{(11)} - 2\alpha \cdot w(v_k, U_l) < 0 \end{aligned} \tag{39}$$

v_l is not affected by the cross-sub-community insertion.

Conclusively, the effects of these edge insertions are:

(1) v_i could be removed from its sub-community only if $\alpha > \frac{4}{\gamma} m - d(I_i)$ or $\alpha > \frac{2w(v_i, U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$ according to **Case 1 and 5**.

(2) v_j could be removed from its sub-community, only if $\alpha > \frac{2w(v_j, U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$ according to **Case 2 and 5**.

(3) $v_k \in S_i \cup S_j$ $(k \neq i, j)$ could be removed from its sub-community only if $\alpha > \frac{w(v_k, U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2} d(U_k)$ according to **Case 3 and 6**.

(4) $v_l \notin S_i \cup S_j$ is unaffected according to **Case 4 and 7**.

□

C INADDTIONAL EXPERIMENTS

• **Effect of γ on modularity and runtime.** Figure 21 shows the average modularity values for all maintenance algorithms, with the parameter $\gamma \in \{0.5, 2, 8, 32\}$ across all 9 batches, and with the batch size fixed at 1000. Across all datasets, our HIT-Leiden still achieves comparable modularity with other methods across different γ. Figure 22 shows the average runtime across all maintenance algorithms under the identical configuration. Across all datasets, the runtime of HIT-Leiden increases as γ increases. This occurs because higher γ values tend to form smaller, finer-grained communities, leading to an increase in the number of super-vertices that change their community or sub-community affiliation.

• **Overall results for AFF.** Figure 23 displays the overall results for the number of supervertices that change their communities or sub-communities(i.e. |AFF|), with b set to its default value $1,000$. Clearly, HIT-Leiden achieves the highest dimensionality reduction on the datasets, particularly on the it-2004 dataset, since it is up to five orders of magnitude faster than the state-of-the-art algorithms.